Artificial Intelligence (AI 302T) — Comprehensive Study Notes

Subject Type: *Theory*
Units Covered: *Unit 1 & Unit 2*
Exam Ready: *Yes*
Course Code: *AI 302T*
University: *B.Tech Syllabus*

Table of Contents

1. Unit 1: Problem Solving & Search
    - 1.1 Introduction to AI
    - 1.2 Problem Solving
    - 1.3 Production Systems
    - 1.4 State Space Search
    - 1.5 Uninformed Search Strategies
    - 1.6 Informed Search Strategies
2. Unit 2: Logic & Knowledge Representation
    - 2.1 Logic Agents
    - 2.2 Knowledge-Based Agents
    - 2.3 Wumpus World
    - 2.4 Propositional Logic
    - 2.5 Inference in Propositional Logic
3. Previous Year Exam Questions

Unit 1: Problem Solving & Search

1.1 Introduction to Artificial Intelligence

What is Artificial Intelligence?

Definition by John McCarthy:

> *"The Science and engineering of making intelligent machines, especially intelligent computer programs."*

Definitions by Other AI Pioneers:

Scholar	Definition
Herbert Simon	"Programs intelligent if they exhibit behaviors that would be regarded intelligent if exhibited by humans"
Elaine Rich	"AI is the study of techniques for solving exponentially hard problems in polynomial time by exploiting knowledge about the problem domain"

What is Intelligence?

Components of Intelligence:

- Ability to calculate and reason
- perceive relationships and analogies
- Store and retrieve information from memory
- Solve problems
- Make decisions
- Classify and generalize
- Learn from experience
- Inductive inference

Types of Intelligence

Type	Description
Linguistic Intelligence	Ability to use language effectively
Musical Intelligence	Ability to understand and create music
Logical-Mathematical Intelligence	Ability to reason logically and solve mathematical problems
Spatial Intelligence	Ability to perceive visual and spatial relationships
Bodily-Kinesthetic Intelligence	Ability to control body movements
Intrapersonal Intelligence	Understanding one's own emotions and motives
Interpersonal Intelligence	Understanding others' emotions and intentions

---

Characteristics of AI Systems

1. Learn new concepts and tasks
2. Reason and draw useful conclusions about the world
3. Remember interrelated facts and draw conclusions (inference)
4. Understand natural language and perceive visual scenes
5. Plan sequences of actions to complete goals
6. Offer advice based on rules and situations
7. Perform tasks that require high levels of intelligence

Research Areas of AI

Area	Description
Problem Solving	Heuristic techniques for solving complex problems
Knowledge Representation	Representing information in computer-readable form
Handling Uncertainty	Probabilistic and fuzzy logic approaches
Theorem Proving	Automated reasoning
Game Playing	Strategic decision making
Natural Language Processing	Understanding and generating human language
Expert Systems	Knowledge-based advisory systems
Robotics	Physical manipulation and navigation

Categories of AI Systems

1. **Systems that think like humans** - Cognitive simulation
2. **Systems that act like humans** - Turing test compliant
3. **Systems that think rationally** - Logic-based
4. **Systems that act rationally** - Rational agents

Turing Test vs Rational Agent Approach

Turing Test Approach (Acting Humanly)

The **Turing Test**, proposed by Alan Turing in 1950, tests a machine's ability to exhibit intelligent behavior equivalent to or indistinguishable from a human.

```
┌──────────────────────────────────────────────────────────────────┐
│                      TURING TEST                                 │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────┐         ┌──────────────┐         ┌──────────┐   │
│    │  HUMAN   │◄───────►│    JUDGE     │◄───────►│ MACHINE  │   │
│    │ (Player  │         │ (Interrogator)         │ (AI)     │   │
│    │    C)    │         │                        │          │   │
│    └──────────┘         └────────────────────────┘          │
│          │                          │                        │
│          │     Communicates via     │                        │
│          │     text terminal        │                        │
│          └──────────────────────────┘                        │
│                                                                  │
│   Goal: Judge cannot distinguish between Human and Machine     │
└──────────────────────────────────────────────────────────────────┘
```

Conditions for Passing Turing Test:

1. Natural language processing
2. Knowledge representation
3. Automated reasoning
4. Machine learning

Criticism of Turing Test:

- Only tests behavior, not intelligence/cognition
- Can be passed by trickery (ELIZA effect)
- Doesn't test visual or physical intelligence
- Human-likeness ≠ Intelligence

Rational Agent Approach (Acting Rationally)

A **Rational Agent** is an agent that acts to achieve the best expected outcome based on its knowledge and percepts.

```
┌──────────────────────────────────────────────────────────────┐
│                    RATIONAL AGENT                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌─────────┐                                               │
│    │PERCEPTS │                                               │
│    └────┬────┘                                               │
│         │                                                    │
│         ▼                                                    │
│    ┌──────────────┐                                          │
│    │    AGENT     │                                          │
│    │  ┌─────────┐ │                                          │
│    │  │ STATE   │ │  ┌──────────────┐  ┌─────────────────┐   │
│    │  │How world│─│─▶│   ACTION     │─▶│  ENVIRONMENT    │   │
│    │  │  is     │ │  │   SELECTOR   │  │  (acts upon)    │   │
│    │  └─────────┘ │  └──────────────┘  └─────────────────┘   │
│    │       │      │        │                                 │
│    │       ▼      │        ▼                                 │
│    │  ┌─────────┐ │   ┌─────────┐                            │
│    │  │KNOWLEDGE│ │   │ACTION   │                            │
│    │  │   BASE  │ │   │         │                            │
│    │  └─────────┘ │   └─────────┘                            │
│    └──────────────┘                                          │
│                                                              │
│   Goal: Select action that maximizes performance measure     │
└──────────────────────────────────────────────────────────────┘
```

Key Components of Rational Agent:

1. **Performance Measure** - Defines what "success" means
2. **Percept Sequence** - Complete history of what agent has perceived
3. **Knowledge** - Information agent has about the environment
4. **Actions** - What agent can do

Difference: Acting Humanly vs Acting Rationally

Aspect	Turing Test Approach	Rational Agent Approach
Goal	Imitate human behavior	Maximize expected performance
Focus	Behavior/Capability	Action/Outcome
Knowledge	May simulate without understanding	Uses explicit knowledge
Method	Test-based	Design-based
Example	Chatbot passing as human	Self-driving car reaching destination

Why Rational Agent is Preferred:

- More well-defined and measurable
- Based on sound mathematical principles
- Not fooled by superficial mimicry
- Can be systematically designed and improved
- Focuses on optimal action, not human-like action

Important Note: Acting rationally ≠ Being perfectly rational. A rational agent selects the best action given its knowledge, but there are limits to what can be computed in reasonable time.

1.2 Problem Solving

What is Problem Solving?

Problem solving in AI involves finding a sequence of actions that transforms an initial state into a goal state.

Key Concepts:

- **State**: A representation of the problem at a point in time
- **Goal State**: The desired end state
- **Operators**: Actions that change one state to another
- **Path**: Sequence of states from initial to goal
- **Cost**: Numerical value associated with path

Requirements of a Good Search Strategy

Requirement	Description
Completeness	Guarantees finding a solution if one exists
Optimality	Finds the best (lowest cost) solution
Time Complexity	How long does it take to find solution
Space Complexity	How much memory is needed

Problem Classification

```
Problems
├── Tractable Problems
│   └── Polynomial time solutions exist (P problems)
├── Intractable Problems
│   └── Exponential time solutions (NP problems)
└── Unsolvable Problems
    └── No solution exists
```

1.3 Production Systems

Definition

A **Production System** is a model of computation consisting of:

- A set of **production rules** (IF-THEN statements)
- A **working memory** (current state/database)
- A **rule interpreter** (control strategy)

Components of Production System

Component	Description
Production Rules	Set of rules in form: IF condition THEN action
Working Memory	Contains current facts about the problem
Rule Interpreter	Decides which rules to apply
Conflict Resolution	When multiple rules apply, which one to choose

Characteristics of Production Systems

1. **Modularity**: Each rule is independent

2. **Modifiability**: Easy to add/modify rules
3. **Incremental**: Knowledge can be added gradually
4. **Data-driven**: Behavior determined by working memory
5. **Search**: System searches through rule applications

Example: Water Jug Problem

Problem: Get exactly 2 liters in the 4-liter jug using 3-liter and 4-liter jugs.

Production Rules:

```
1. IF (X < 4) THEN fill_4L_jug
2. IF (Y < 3) THEN fill_3L_jug
3. IF (X > 0) THEN empty_4L_jug
4. IF (Y > 0) THEN empty_3L_jug
5. IF (X > 0 AND Y < 3) THEN pour_4L_to_3L
6. IF (Y > 0 AND X < 4) THEN pour_3L_to_4L
```

1.4 State Space Search

What is State Space?

State Space is the set of all possible states that can be reached from the initial state through any sequence of operators.

```
State Space = All reachable states from initial state
```

State Representation

A state can be represented as:

- **Atomic**: Single value (e.g., state = "success")
- **Factored**: Vector of attributes (e.g., [location, hasGold, direction])
- **Structured**: Complex objects with relationships

Example: 8-Puzzle State

```
Initial State:     Goal State:
+---+---+---+     +---+---+---+
| 1 | 2 | 3 |     | 1 | 2 | 3 |
+---+---+---+     +---+---+---+
| 4 | 5 | 6 |     | 4 | 5 | 6 |
+---+---+---+     +---+---+---+
| 7 |   | 8 |     |   | 7 | 8 |
+---+---+---+     +---+---+---+
```

State: Array [1,2,3,4,5,6,7,0,8] where 0 is empty

1.5 Uninformed Search Strategies (Blind Search)

Overview

Uninformed Search (Blind Search) algorithms have no information about the goal except for the problem definition. They explore the search tree systematically.

---

1. Breadth-First Search (BFS)

BFS explores all nodes at depth d before exploring nodes at depth d+1.

```python
"""
BFS (Breadth-First Search) Algorithm - Complete Explanation

KEY CONCEPT:
- Explores ALL nodes at current depth BEFORE moving to next depth
- Uses FIFO Queue (First In First Out)
- Guarantees finding SHALLOWEST (shortest) solution

WHY BFS WORKS:
- Level by level exploration ensures shortest path
- All nodes at depth d are processed before any node at d+1

COMPLETENESS: Yes (if solution exists)
OPTIMALITY: Yes (finds shortest path in terms of edges)
TIME COMPLEXITY: O(b^d) where b=branching factor, d=depth
SPACE COMPLEXITY: O(b^d)
"""

from collections import deque

def bfs(graph, start, goal):
    """
    Breadth-First Search Algorithm

    BFS explores nodes level by level using a FIFO queue.
    Guarantees finding the shortest path (fewest edges) to goal.

    Args:
        graph: Dictionary representing adjacency list
               Example: {'A': ['B', 'C'], 'B': ['D', 'E']}
        start: Starting node
        goal: Goal node to find

    Returns:
        Path from start to goal as list, or None if not found
    """
    # Step 1: Initialize queue with starting node
    # Queue stores: (current_node, path_to_node)
    # We track path to reconstruct solution
    queue = deque([(start, [start])])

    # Step 2: Initialize visited set
    # Prevents revisiting nodes (avoids infinite loops)
    visited = set()

    # Step 3: Main loop - continue until queue empty or goal found
    while queue:
        # Step 4: Dequeue - remove from FRONT (oldest added)
        # This is BFS key: processes nodes in order they were added
        node, path = queue.popleft()

        # Step 5: Check if goal reached
        if node == goal:
            return path

        # Step 6: Skip if already visited
        if node in visited:
            continue

        # Step 7: Mark as visited
```

```python
        visited.add(node)

        # Step 8: Add all unvisited neighbors to queue
        # They will be processed after all current level nodes
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))

    # No solution found
    return None


# Example usage
graph = {
    'S': ['A', 'B'],  # S = Start
    'A': ['C', 'D'],  # A, B = depth 1
    'B': ['E', 'F'],  # C, D, E, F = depth 2
    'C': ['G'],       # G = depth 3
    'D': ['G'],
    'E': [],
    'F': ['K'],       # K = Goal at depth 3
    'G': [],
    'K': []
}

result = bfs(graph, 'S', 'K')
print(f"BFS Path: {result}")  # Output: ['S', 'B', 'F', 'K']
```

---

2. Depth-First Search (DFS)

DFS explores as deep as possible along each branch before backtracking.

```python
"""
DFS (Depth-First Search) Algorithm - Complete Explanation

KEY CONCEPT:
- Explores DEEP into one branch before backtracking
- Uses Stack (LIFO) or recursion
- Does NOT guarantee shortest path

WHY DFS WORKS:
- Goes deep along a path until dead-end
- When stuck, backtracks to try other paths
- Much less memory than BFS

COMPLETENESS: Yes (in finite spaces without cycles)
OPTIMALITY: NO (may find longer path first)
TIME COMPLEXITY: O(b^m) where m = maximum depth
SPACE COMPLEXITY: O(bm)
"""

def dfs(graph, start, goal, visited=None, path=None):
    """
    Depth-First Search Algorithm

    DFS uses recursion to explore deeply before backtracking.
    Does NOT guarantee shortest path but uses less memory.

    Args:
        graph: Dictionary representing adjacency list
        start: Starting node
        goal: Goal node to find
        visited: Set of visited nodes (shared across recursive calls)
        path: Current path from start to current node

    Returns:
        Path from start to goal, or None if not found
    """
    # Step 1: Initialize on first call
    if visited is None:
        visited = set()
    if path is None:
        path = []

    # Step 2: Add current node to path and visited
    path = path + [start]
    visited.add(start)

    # Step 3: Check if goal reached
    if start == goal:
        return path

    # Step 4: Recursively explore neighbors
    # Key DFS: processes first neighbor completely before second
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path)
            if result:
                return result

    # Step 5: Backtrack - return None to try other branches
    return None
```

```python
# Example usage - same graph as BFS
result = dfs(graph, 'S', 'K')
print(f"DFS Path: {result}")  # Output: ['S', 'A', 'C', 'G']
```

3. Depth-Limited Search (DLS)

Sets a maximum depth limit to solve infinite depth problem of DFS.

```python
"""
Depth-Limited Search (DLS) Algorithm

KEY CONCEPT:
- Like DFS but with a depth limit ℓ
- Nodes at depth limit are treated as having no successors
- Solves infinite path problem of DFS
"""

def depth_limited_search(graph, start, goal, limit, visited=None):
    """DFS with depth limit"""
    if visited is None:
        visited = set()

    # Check if goal at current depth
    if start == goal:
        return [start]

    # Check if limit reached - cannot go deeper
    if limit <= 0:
        return 'cutoff'

    visited.add(start)

    # Explore neighbors with reduced limit
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            result = depth_limited_search(graph, neighbor, goal,
                                          limit-1, visited)
            if result and result != 'cutoff':
                return [start] + result

    return None
```

4. Iterative Deepening DFS (IDDFS)

Combines benefits of BFS and DFS by trying increasing depth limits.

```python
"""
Iterative Deepening Depth-First Search (IDDFS)

KEY CONCEPT:
- Repeats DLS with increasing depth limits
- Combines BFS completeness with DFS memory efficiency

ADVANTAGES:
- Complete (finds solution if exists)
- Optimal (finds shortest path like BFS)
- Memory efficient (like DFS)

TIME: O(b^d)
SPACE: O(bd)
"""

def iterative_deepening_dfs(graph, start, goal, max_depth=100):
    """
    IDDFS - Combines BFS and DFS benefits

    Args:
        graph: Graph representation
        start: Starting node
        goal: Goal node
        max_depth: Maximum depth to search

    Returns:
        Path if found, None otherwise
    """
    # Step 1: Try with depth 0, 1, 2, ... until found
    for depth in range(max_depth):
        result = depth_limited_search(graph, start, goal, depth)
        if result and result != 'cutoff':
            return result
    return None
```

---

5. Uniform Cost Search (UCS)

Finds path with minimum cumulative cost (for weighted graphs).

```python
"""
Uniform Cost Search (UCS) - Complete Explanation

KEY CONCEPT:
- Finds path with MINIMUM CUMULATIVE COST
- Uses Priority Queue (min-heap) ordered by path cost
- Unlike BFS (counts edges), UCS considers edge weights

DIFFERENCE FROM BFS:
- BFS: Explores by number of edges (unweighted)
- UCS: Explores by cumulative cost (weighted)

GUARANTEES: Optimal (least-cost) solution
"""

import heapq

def uniform_cost_search(graph, start, goal):
    """
    Uniform Cost Search Algorithm

    UCS finds path with minimum total cost.
    Uses priority queue ordered by cumulative cost.

    Args:
        graph: Dictionary with costs
               Format: {'A': [('B', 5), ('C', 2)]}
        start: Starting node
        goal: Goal node

    Returns:
        (path, total_cost) or None
    """
    # Priority queue: (cost, node, path)
    priority_queue = [(0, start, [start])]
    visited = set()

    while priority_queue:
        # Step 1: Get node with LOWEST cumulative cost
        cost, node, path = heapq.heappop(priority_queue)

        # Step 2: Skip if visited
        if node in visited:
            continue

        # Step 3: Mark as visited
        visited.add(node)

        # Step 4: Check if goal
        if node == goal:
            return path, cost

        # Step 5: Explore neighbors with cost
        for neighbor, edge_cost in graph.get(node, []):
            if neighbor not in visited:
                new_cost = cost + edge_cost
                heapq.heappush(priority_queue,
                               (new_cost, neighbor, path + [neighbor]))

    return None
```

```python
# Example with costs
graph = {
    'S': [('A', 1), ('B', 2)],
    'A': [('C', 1), ('D', 3)],
    'B': [('F', 1)],
    'C': [('G', 2)],
    'D': [('G', 1)],
    'F': [('K', 3)],
    'G': [],
    'K': []
}

result = uniform_cost_search(graph, 'S', 'K')
print(f"UCS: {result}")  # Returns optimal cost path
```

Comparison of Uninformed Search

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes	Yes (shallowest)	O(b^d)	O(b^d)
DFS	No	No	O(b^m)	O(bm)
DLS	No	No	O(bℓ)	O(bℓ)
IDDFS	Yes	Yes	O(b^d)	O(bd)
UCS	Yes	Yes	O(b^(1+C*/ε))	O(b^(1+C*/ε))

1.6 Informed Search Strategies (Heuristic Search)

What are Heuristics?

Heuristic is a technique that uses domain-specific knowledge to find solutions faster than uninformed methods.

```
Heuristic h(n) = Estimated cost from node n to goal
```

---

1. Best-First Search (Greedy Search)

Expands node that appears closest to goal based on heuristic.

```python
"""
Best-First Search (Greedy Search) - Complete Explanation

KEY CONCEPT:
- Expands node with LOWEST heuristic value h(n)
- Uses ONLY heuristic (ignores path cost)
- Formula: f(n) = h(n)

NOT OPTIMAL - May find non-shortest path
"""

import heapq

def greedy_best_first_search(graph, start, goal, heuristic):
    """
    Greedy Best-First Search

    Expands node with lowest h(n) value (closest to goal)

    Args:
        graph: Adjacency list
        start: Starting node
        goal: Goal node
        heuristic: Dictionary of h(n) values
                   Example: {'S': 6, 'A': 4, 'K': 0}

    Returns:
        Path from start to goal
    """
    # Priority queue: (heuristic_value, node, path)
    priority_queue = [(heuristic[start], start, [start])]
    visited = set()

    while priority_queue:
        # Get node with LOWEST h(n)
        _, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return path

        # Add neighbors ordered by heuristic value
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(priority_queue,
                               (heuristic[neighbor], neighbor,
                                path + [neighbor]))

    return None
```

2. A* Search Algorithm

A* combines actual cost from start (g) with heuristic to goal (h).

```python
"""
A* Search Algorithm - Complete Explanation

KEY CONCEPT:
- Combines g(n) + h(n) for optimal path finding
- Most famous heuristic search algorithm!
- Guarantees optimal solution with ADMISSIBLE heuristic

FORMULA:
f(n) = g(n) + h(n)

WHERE:
- g(n) = Actual cost from START to node n
- h(n) = Estimated cost from n to GOAL
- f(n) = Total estimated cost (start → n → goal)

ADMISSIBLE HEURISTIC:
- Never overestimates the actual cost
- Optimistic (thinks it's easier than it is)
- h(n) ≤ h*(n) where h* is true cost
"""

import heapq

def a_star_search(graph, start, goal, heuristic):
    """
    A* Search Algorithm

    Combines actual cost g(n) with heuristic h(n) to find optimal path.
    Guarantees optimal solution with admissible heuristic.

    Args:
        graph: Dictionary with edge costs
               Format: {'A': [('B', 5), ('C', 2)]}
        start: Starting node
        goal: Goal node
        heuristic: Dictionary of h(n) values (estimated cost to goal)

    Returns:
        (path, total_cost) or None
    """
    # Priority queue: (f_score, g_score, node, path)
    # f = g + h
    open_set = [(heuristic[start], 0, start, [start])]
    closed_set = set()

    while open_set:
        # Step 1: Pop node with LOWEST f(n)
        f, g, node, path = heapq.heappop(open_set)

        # Step 2: Skip if already processed
        if node in closed_set:
            continue

        # Step 3: Mark as processed
        closed_set.add(node)

        # Step 4: Check if goal reached
        if node == goal:
            return path, g
```

```python
        # Step 5: Explore neighbors
        for neighbor, cost in graph.get(node, []):
            if neighbor not in closed_set:
                # Calculate new g (cost to reach neighbor)
                new_g = g + cost
                # Calculate new f = g + h
                new_f = new_g + heuristic.get(neighbor, float('inf'))
                heapq.heappush(open_set,
                             (new_f, new_g, neighbor,
                              path + [neighbor]))

    return None


# ==================== EXAMPLE ====================
"""
Graph with costs and heuristics:

        S(0)
       /   \
   A(1)/    \B(2)
     /       \
   C(1)     F(1)
    |         |
   G(2)      K(3)  (Goal)

Heuristic h(n): Estimated cost from node to goal K
S=6, A=4, B=5, C=3, D=4, F=4, G=3, K=0

Step-by-step A* execution:
1. OPEN: {S(f=6,g=0)}
2. Expand S: Add A(f=4,g=1), B(f=7,g=2)
3. Expand A: Add C(f=4,g=2), D(f=8,g=4)
4. Expand C: Add G(f=5,g=3)
5. Expand G: Add K(f=5,g=5) → GOAL!

Optimal Path: S → A → C → G → K (Cost: 5)
"""
```

Properties of A*

Property	Description
Completeness	Yes, will find solution if exists
Optimality	Yes, with admissible heuristic
Time	O(b^d) in worst case
Space	O(b^d) - keeps all nodes in memory

3. Hill Climbing Search

Greedy local search that moves to neighbor with highest value.

```python
"""
Hill Climbing Algorithm - Complete Explanation

KEY CONCEPT:
- Local search algorithm
- Moves to neighbor with BEST value (highest for maximization)
- Does NOT maintain search tree
- Terminates at LOCAL MAXIMUM (not necessarily global)

VARIANTS:
1. Simple Hill Climbing: Evaluate one neighbor at a time
2. Steepest-Ascent Hill Climbing: Evaluate ALL neighbors, pick best
3. Stochastic Hill Climbing: Random selection with probability
"""

def hill_climbing(problem):
    """
    Simple Hill Climbing Algorithm

    Args:
        problem: Object with methods:
                 - initial_state()
                 - get_successors()
                 - value(state)

    Returns:
        Best state found
    """
    # Step 1: Start with initial state
    current = problem.initial_state()

    while True:
        # Step 2: Get all neighbors
        neighbors = current.get_successors()

        # Step 3: Find best neighbor
        best_neighbor = None
        best_value = current.value()

        for neighbor in neighbors:
            if neighbor.value() > best_value:
                best_neighbor = neighbor
                best_value = neighbor.value()

        # Step 4: If no better neighbor, we're done
        if best_value <= current.value():
            return current

        # Step 5: Move to better neighbor
        current = best_neighbor


"""
Problems in Hill Climbing:

1. LOCAL MAXIMUM:
   - Reached peak lower than global maximum
   - Solution: Random restart, backtracking

2. PLATEAU:
```

```
   - Flat area where all neighbors have same value
   - Solution: Big jump, random move

3. RIDGE:
   - Steep area but can only reach in certain directions
   - Solution: Try multiple directions, bidirectional search
"""
```

Comparison: Informed vs Uninformed Search

Aspect	Uninformed	Informed
Knowledge	No domain knowledge	Uses heuristic h(n)
Efficiency	Less efficient	More efficient
Optimal	Sometimes	Always (with good h)
Examples	BFS, DFS, UCS	A*, Greedy, Hill Climbing

1.7 Adversarial Search (Game Playing)

What is Adversarial Search?

In games where there are **multiple agents** (opponents) with conflicting goals, we need adversarial search techniques. Unlike single-agent search, we must consider that the opponent will try to minimize our success.

```
┌──────────────────────────────────────────────────────────────────┐
│                 TWO-PLAYER ZERO-SUM GAME                         │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│                      MAX (Our Player)                            │
│                          ┌─┐                                     │
│                          │X│  ↑  Wants MAXIMIZE score          │
│                          └─┘                                     │
│                        /  |  \                                   │
│                   ┌───┐   |   ┐───┐                              │
│                  /   \   |  /   \                               │
│                 ▼     ▼  | ▼     ▼                              │
│               ┌─────┐  ┌─────┐  ┌─────┐                         │
│               │State│  │State│  │State│                          │
│               │ A   │  │ B   │  │ C   │                          │
│               └─────┘  └─────┘  └─────┘                         │
│                  |        |        |                             │
│                  ▼        ▼        ▼                             │
│                      MIN (Opponent)                              │
│                          ┌─┐                                     │
│                          │O│  ↓  Wants MINIMIZE score          │
│                          └─┘                                     │
│                                                                  │
│   MAX tries to get highest score                                 │
│   MIN tries to get lowest score                                  │
└──────────────────────────────────────────────────────────────────┘
```

Minimax Algorithm

Minimax is a recursive algorithm used in two-player zero-sum games where:

- **MAX** wants to maximize the score
- **MIN** wants to minimize the score

```python
"""
Minimax Algorithm - Complete Explanation

KEY CONCEPT:
- Two players: MAX and MIN
- MAX maximizes score, MIN minimizes
- Recursively explores all possible moves
- Returns best score assuming optimal opponent

COMPLETENESS: Yes (for finite games)
TIME COMPLEXITY: O(b^m) where b=branching, m=depth
SPACE COMPLEXITY: O(bm)
"""

import math

def minimax(node, depth, is_maximizing, game_tree):
    """
    Minimax Algorithm

    Args:
        node: Current game state
        depth: Current depth in game tree
        is_maximizing: True if MAX's turn, False for MIN
        game_tree: Game tree with scores at leaves

    Returns:
        Best score achievable from this node
    """
    # BASE CASE: Leaf node (terminal state)
    if is_leaf(node, game_tree):
        return get_utility(node, game_tree)

    # RECURSIVE CASE
    if is_maximizing:
        best_score = -math.inf
        for child in get_children(node, game_tree):
            score = minimax(child, depth + 1, False, game_tree)
            best_score = max(best_score, score)
        return best_score
    else:
        best_score = math.inf
        for child in get_children(node, game_tree):
            score = minimax(child, depth + 1, True, game_tree)
            best_score = min(best_score, score)
        return best_score
```

Alpha-Beta Pruning

Alpha-Beta Pruning optimizes Minimax by eliminating branches that cannot influence the final decision.

```
┌──────────────────────────────────────────────────────────────────┐
│                  ALPHA-BETA PRUNING                              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ALPHA: Best value MAX can guarantee at current path          │
│   BETA:  Best value MIN can guarantee at current path          │
│                                                                  │
│   PRUNING RULES:                                                 │
│   - If beta ≤ alpha: Prune remaining siblings                  │
│                                                                  │
│   Example Tree:                                                  │
│                                                                  │
│                        MAX                                       │
│                       /   \                                      │
│                  (α=3)   Branch B ──┐                            │
│                     │              │                            │
│                  Branch A         │                            │
│                     │              │                            │
│               ┌─────┴──────┐      │                            │
│               │            │      │                            │
│             MIN           MIN    MIN  (pruned!)                 │
│             / \            │      │                            │
│            2   5    ← β≤α  Pruned                              │
│                                                                  │
│   We don't need to explore MIN's children after seeing 2        │
│   because we already have a better option (3)                  │
└──────────────────────────────────────────────────────────────────┘
```

```python
"""
Alpha-Beta Pruning

KEY CONCEPT:
- Eliminates unnecessary exploration
- Same result as Minimax, much faster
- Alpha = best value MAX can achieve
- Beta = best value MIN can achieve

TIME COMPLEXITY: O(b^(m/2)) for good ordering
"""

import math

def alpha_beta(node, depth, alpha, beta, is_maximizing, game_tree):
    if is_leaf(node, game_tree):
        return get_utility(node, game_tree)

    if is_maximizing:
        max_eval = -math.inf
        for child in get_children(node, game_tree):
            eval_score = alpha_beta(child, depth + 1, alpha, beta,
                                    False, game_tree)
            max_eval = max(max_eval, eval_score)
            alpha = max(alpha, eval_score)
            if beta <= alpha:
                break  # PRUNING
        return max_eval
    else:
        min_eval = math.inf
        for child in get_children(node, game_tree):
            eval_score = alpha_beta(child, depth + 1, alpha, beta,
                                    True, game_tree)
            min_eval = min(min_eval, eval_score)
            beta = min(beta, eval_score)
            if beta <= alpha:
                break  # PRUNING
        return min_eval
```

Comparison: Minimax vs Alpha-Beta

Aspect	Minimax	Alpha-Beta
Time Complexity	O(b^m)	O(b^(m/2)) ideal
Completeness	Yes	Yes
Optimality	Yes	Yes
Pruning	None	Eliminates branches

1.8 Constraint Satisfaction Problems (CSP)

What is CSP?

A **Constraint Satisfaction Problem** is defined by:

- A set of **variables** (X_1, X_2, ..., X_n)
- Each variable has a **domain** (possible values)
- A set of **constraints** (rules about allowed values)

```
┌──────────────────────────────────────────────────────────────────┐
│              CONSTRAINT SATISFACTION PROBLEM                      │
├──────────────────────────────────────────────────────────────────┤
│   COMPONENTS:                                                     │
│   Variables: X, Y, Z                                              │
│   Domains:   {1, 2, 3}                                            │
│   Constraints: X≠Y, X<Y, X+Y=Z                                   │
│                                                                   │
│   GOAL: Find assignment that satisfies ALL constraints            │
└──────────────────────────────────────────────────────────────────┘
```

Types of Constraints

Constraint Type	Example	Description
Unary	X ≠ 5	Single variable
Binary	X ≠ Y	Two variables
Global	AllDifferent(X,Y,Z)	Multiple variables

Example 1: Map Coloring

```
Problem: Color map with 3 colors (adjacent = different color)

Variables: WA, NT, Q, NSW, V, SA, T
Domains:   {Red, Green, Blue}

Solution:
┌──────────────────────────────────────┐
│   WA(Red)──NT(Blue)                  │
│      │          │                    │
│      │    SA(Green)                 │
│      │    /   \                     │
│   Q(Red)    NSW(Red)                 │
│      │       │                      │
│      │    V(Blue)                   │
│      │                              │
│      T(Any)                         │
└──────────────────────────────────────┘
```

Example 2: N-Queens Problem

```
Problem: Place N queens on N×N chessboard (no attacks)

For N=4:
    1   2   3   4
  ┌───┬───┬───┬───┐
1 │   │ Q │   │   │  Q at (1,2)
  ├───┼───┼───┼───┤
2 │   │   │   │ Q │  Q at (2,4)
  ├───┼───┼───┼───┤
3 │ Q │   │   │   │  Q at (3,1)
  ├───┼───┼───┼───┤
4 │   │   │ Q │   │  Q at (4,3)
  └───┴───┴───┴───┘
```

Backtracking Search for CSP

```python
"""
Backtracking Search for CSP
"""

def backtracking_search(csp):
    return backtrack({}, csp)

def backtrack(assignment, csp):
    if is_complete(assignment, csp):
        return assignment

    var = select_unassigned_variable(assignment, csp)

    for value in order_domain_values(var, assignment, csp):
        if is_consistent(var, value, assignment, csp):
            assignment[var] = value
            result = backtrack(assignment, csp)
            if result is not None:
                return result
            del assignment[var]

    return None
```

CSP Heuristics

Heuristic	Description
MRV	Minimum Remaining Values - choose most constrained variable
LCV	Least Constraining Value - choose value that leaves most options

---

Unit 2: Logic & Knowledge Representation

2.1 Logic Agents

What is Logic?

Definition:

> *Logic is the scientific study of the process of reasoning and system of rules and procedures that help in the reasoning process.*

Types of Logic

Type	Description
Philosophical Logic	500 BC - 19th Century, dealt with natural language arguments
Propositional Logic	Deals with true/false statements
Predicate Logic	Deals with objects and their relationships
First-Order Logic	Extends predicate logic with quantifiers

Logic in Computer Science

Application	Use of Logic
Circuit Design	Propositional logic for boolean circuits
Databases	Query languages (SQL) use relational logic
Programming	Prolog uses logic programming
Verification	Design validation and model checking
AI	Inference systems, expert systems

2.2 Knowledge-Based Agents

What is a Knowledge-Based Agent?

A **Knowledge-Based Agent** uses knowledge representation and reasoning to make decisions.

```
┌──────────────────────────────────────────┐
│           Knowledge-Based Agent          │
├──────────────────────────────────────────┤
│  ┌──────────────┐    ┌──────────────────┐ │
│  │ Knowledge    │    │     Inference    │ │
│  │    Base      │ ←→ │      Engine      │ │
│  └──────────────┘    └──────────────────┘ │
│         ↑                    ↓           │
│  ┌──────────────────────────────────────┐ │
│  │        TELL / ASK Interface          │ │
│  └──────────────────────────────────────┘ │
└──────────────────────────────────────────┘
```

Components

Component	Description
Knowledge Base (KB)	Set of sentences representing knowledge
Inference Engine	Derives new conclusions from KB
TELL	Adds new knowledge to KB
ASK	Queries the KB
Explanation Module	Provides reasoning behind conclusions

Knowledge Representation

Requirements of Good Representation:

1. **Representational Adequacy** - Can represent all needed knowledge
2. **Inferential Adequacy** - Can derive new knowledge
3. **Inferential Efficiency** - Can derive knowledge efficiently
4. **Expressiveness** - Can express knowledge clearly

2.3 Wumpus World

What is Wumpus World?

The **Wumpus World** is a classic AI problem used to demonstrate knowledge-based agents.

PEAS Description

Component	Description
Performance	Gold: +1000, Death: -1000, -1 per step
Environment	Grid of rooms, smelly near Wumpus, breezy near pits
Actuators	Left, Right, Forward, Grab, Release, Shoot
Sensors	Stench, Breeze, Glitter, Bump, Scream

Wumpus World Characteristics

Property	Value
Observability	Partial (only local perception)
Deterministic	Yes (outcomes exactly specified)
Episodic	No (sequential at action level)
Static	Yes (wumpus doesn't move)
Discrete	Yes
Agent	Single

Wumpus World Rules

1. **Stench** - Adjacent to Wumpus
2. **Breeze** - Adjacent to pit
3. **Glitter** - Gold in same square
4. **Wumpus** - Deadly, can be killed
5. **Pit** - Deadly, cannot be escaped
6. **Arrow** - Can kill Wumpus, only one shot

2.4 Propositional Logic

What is Propositional Logic?

Propositional Logic (or Boolean Logic) deals with propositions that are either TRUE or FALSE.

Syntax

Symbol	Meaning	Example
P, Q, R	Propositional symbols	"It is raining"
¬	Negation (NOT)	¬P = "It is not raining"
∧	Conjunction (AND)	P ∧ Q = "Raining AND cold"
∨	Disjunction (OR)	P ∨ Q = "Raining OR cold"
→	Implication	P → Q = "If raining, then cold"
↔	Biconditional	P ↔ Q = "Raining iff clouds"

Truth Tables

P	Q	¬P	P∧Q	P∨Q	P→Q	P↔Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Logical Equivalences

Equivalence	Formula
Identity	$P \leftrightarrow P$
Double Negation	$\neg\neg P \leftrightarrow P$
De Morgan's Law 1	$\neg(P \land Q) \leftrightarrow (\neg P \lor \neg Q)$
De Morgan's Law 2	$\neg(P \lor Q) \leftrightarrow (\neg P \land \neg Q)$
Implication	$P \rightarrow Q \leftrightarrow \neg P \lor Q$
Contrapositive	$P \rightarrow Q \leftrightarrow \neg Q \rightarrow \neg P$

2.5 Inference in Propositional Logic

What is Inference?

Inference is the process of deriving new facts from known facts using inference rules.

Forward Chaining

Forward Chaining (Data-Driven):

- Starts from known facts
- Applies rules to derive new facts
- Continues until goal reached

```python
"""
Forward Chaining - Complete Explanation

KEY CONCEPT:
- DATA-DRIVEN reasoning
- BOTTOM-UP: From facts to conclusions
- Starts with known facts in KB
- Applies Modus Ponens to derive new facts
- Continues until goal is derived or no more facts

USED IN: Expert systems, production systems, deductive databases

"""

def forward_chaining(KB, goal):
    """
    Forward Chaining Algorithm

    Args:
        KB: Knowledge Base with facts and rules
        goal: Goal predicate to prove

    Returns:
        True if goal can be derived
    """
    # Step 1: Start with known facts
    inferred = set(KB['facts'])
    new_facts = set(KB['facts'])

    # Step 2: Main loop - continue until no new facts
    while new_facts:
        new_inferred = set()

        # Step 3: For each rule, try to apply
        for rule in KB['rules']:
            # Rule format: (premise1, premise2, ...) → conclusion
            for fact in new_facts:
                # Step 4: Try to match premise with fact
                if match(rule.premise, fact):
                    # Step 5: Derive conclusion
                    conclusion = apply(rule.conclusion, fact)
                    if conclusion not in inferred:
                        new_inferred.add(conclusion)

        # Step 6: Update facts
        new_facts = new_inferred
        inferred.update(new_facts)

        # Step 7: Check if goal reached
        if goal in inferred:
            return True

    return False
```

Backward Chaining

Backward Chaining (Goal-Driven):

- Starts from goal
- Works backwards to find supporting facts
- More focused than forward chaining

```python
"""
Backward Chaining - Complete Explanation

KEY CONCEPT:
- GOAL-DRIVEN reasoning
- TOP-DOWN: From goal to facts
- Starts with what we want to prove
- Works backwards to find supporting facts
- More efficient - only relevant facts explored

USED IN: Prolog, diagnostic systems, question-answering
"""

def backward_chaining(KB, goal, visited=None):
    """
    Backward Chaining Algorithm

    Args:
        KB: Knowledge Base
        goal: Goal to prove
        visited: Set of goals being proved (cycle detection)

    Returns:
        True if goal can be proved
    """
    if visited is None:
        visited = set()

    # Step 1: Check for cycles
    if goal in visited:
        return False

    visited.add(goal)

    # Step 2: Check if goal is a known fact
    if goal in KB['facts']:
        return True

    # Step 3: Find rules that can conclude goal
    for rule in KB.get_rules_for(goal):
        # Step 4: Try to prove all premises
        all_proved = True
        for premise in rule.premises:
            if not backward_chaining(KB, premise, visited):
                all_proved = False
                break

        if all_proved:
            return True

    return False
```

Resolution

Resolution is a complete theorem-proving technique using proof by contradiction.

```
"""
Resolution - Complete Explanation

KEY CONCEPT:
- REFUTATION theorem proving
- PROOF BY CONTRADICTION
- To prove KB ⊨ query:
  1. Assume ¬query (negate conclusion)
  2. Add to KB
  3. Derive contradiction (empty clause)
  4. If contradiction → query is TRUE

RESOLUTION RULE:
  (L₁ ∨ L₂ ∨ ... ∨ Lₙ)  and  (¬L₁ ∨ M)
  ──────────────────────────────────────
  L₂ ∨ ... ∨ Lₙ ∨ M

Steps:
1. Convert to CNF
2. Negate conclusion
3. Resolve until empty clause
4. Empty clause = contradiction = PROOF!
"""

def resolution(KB, query):
    """
    Resolution Theorem Proving

    Args:
        KB: Knowledge Base
        query: Statement to prove

    Returns:
        True if KB entails query
    """
    # Step 1: Negate the query
    negated = negate(query)

    # Step 2: Add to KB (in CNF)
    clauses = KB.clauses + to_cnf(negated)

    # Step 3: Resolution loop
    while True:
        new_clauses = []

        # Step 4: Try all pairs
        for i, ci in enumerate(clauses):
            for cj in clauses[i+1:]:
                resolvent = resolve(ci, cj)

                # Step 5: Empty clause = contradiction = PROOF!
                if is_empty(resolvent):
                    return True

                if resolvent:
                    new_clauses.append(resolvent)

        # Step 6: Check for progress
        if not new_clauses:
            return False
```

```
        # Step 7: Add new clauses
        clauses.extend(new_clauses)
```

Resolution Example

Problem: Given facts and rules, prove a conclusion.

```
KB:
1. P → Q
2. P

Prove: Q

Step 1: CNF form
- ¬P ∨ Q
- P
- ¬Q (negate conclusion)

Step 2: Resolution
¬P ∨ Q  +  P  →  Q  (resolve P and ¬P)
Q  +  ¬Q  →  ⊥  (empty clause!)

CONTRADICTION! Therefore Q is TRUE ✓
```

Comparison: Forward vs Backward Chaining vs Resolution

Aspect	Forward Chaining	Backward Chaining	Resolution
Direction	Facts → Goal	Goal → Facts	Goal added to KB
Method	Bottom-up	Top-down	Refutation
Control	Data-driven	Goal-driven	Systematic
Complete	Yes	Yes	Yes
Used In	Expert Systems	Prolog	Theorem Provers

Previous Year Exam Questions

Question Paper - 1

Q1 (a) What is AI? Differentiate between intelligent behavior and intelligent action. (3 marks)

Answer:

```
AI (Artificial Intelligence) is the study of making machines intelligent.

Intelligent Behavior:
- Actions based on reasoning
- Learning from experience
- Adapting to new situations
- Understanding complex concepts

Intelligent Action:
- Performing tasks that achieve goals
- Effective and efficient completion
- May not involve understanding

Key Difference: Behavior is about HOW (process), Action is about WHAT (result)
```

Q1 (b) Write steps for BFS algorithm. (3 marks)

Answer:

```
Breadth-First Search Algorithm Steps:

1. Initialize: Create queue, add start node with its path
2. Create visited set to track processed nodes
3. While queue is not empty:
   a. Dequeue node from front
   b. If node is goal, return path
   c. If node not visited:
      - Mark as visited
      - Add all unvisited neighbors to queue
4. If queue empty and goal not found, return failure
```

Q1 (c) Explain heuristic function. How it is useful in heuristic search? (4 marks)

Answer:

```
Heuristic Function h(n):

Definition: Estimated cost from node n to goal node
- Provides "educated guess" about remaining cost
- Domain-specific knowledge
- Guides search toward goal

Formula in A* Search:
f(n) = g(n) + h(n)

Where:
- g(n) = Actual cost from start to n
- h(n) = Estimated cost from n to goal
- f(n) = Total estimated cost

How Heuristics Help:
1. Reduce search space significantly
2. Find solutions faster than blind search
3. With admissible heuristic, guarantees optimal solution
4. Prioritizes promising nodes

Example: In 8-puzzle, h(n) = number of misplaced tiles
```

Q2 (a) Compare informed and uninformed search. (5 marks)

Answer:

Criterion	Uninformed Search	Informed Search
Other Name	Blind Search	Heuristic Search
Knowledge	No domain knowledge	Uses heuristic h(n)
Examples	BFS, DFS, UCS	A*, Greedy, Hill Climbing
Efficiency	Less efficient	More efficient
Optimal	Sometimes (BFS, UCS)	With good heuristic
Completeness	Usually yes	Usually yes
Memory	High	Moderate to High

Q2 (b) Explain A* search algorithm with example. (5 marks)

Answer:

```
A* Search Algorithm:

Formula: f(n) = g(n) + h(n)

Where:
- g(n) = Actual cost from start to n
- h(n) = Heuristic estimate from n to goal
- f(n) = Total estimated cost

Example Graph:

        S(0)
       /   \
   A(1)/    \B(2)
     /       \
   C(1)     F(1)
    |         |
   G(2)      K(3)  Goal

Heuristic h(n): S=6, A=4, B=5, C=3, F=4, G=3, K=0

Execution:
1. Start: OPEN = {S(f=6)}
2. Expand S: OPEN = {A(f=4), B(f=7)}
3. Expand A: OPEN = {C(f=4), B(f=7), D(f=8)}
4. Expand C: OPEN = {G(f=5), B(f=7), D(f=8)}
5. Expand G: Goal K found! Path: S→A→C→G→K

Optimal: Yes (with admissible heuristic)
```

Q3 (a) Explain production system with characteristics. (5 marks)

Answer:

```
Production System:

Definition: A computational model with:
- Production rules (IF-THEN statements)
- Working memory (current facts)
- Rule interpreter (control strategy)

Characteristics:

1. Modularity: Each rule is independent
2. Modifiability: Easy to add/modify/delete rules
3. Incremental: Knowledge can be added gradually
4. Data-driven: Behavior determined by working memory
5. Search: System searches through possible rule applications
6. Separation: Rules are separate from control mechanism

Example: Water Jug Problem
IF jug1 < 4 THEN fill_jug1
IF jug2 < 3 THEN fill_jug2
IF jug1 > 0 AND jug2 < 3 THEN pour_jug1_to_jug2
```

Q3 (b) Explain Hill Climbing algorithm with its problems. (5 marks)

Answer:

```
Hill Climbing Algorithm:

Approach:
1. Start with initial state
2. Evaluate current state
3. Move to better neighbor
4. Repeat until no better neighbor

Types:
- Simple: Check one neighbor at a time
- Steepest-Ascent: Check all neighbors, pick best
- Stochastic: Random selection

Problems:

1. LOCAL MAXIMUM:
   - Reached peak lower than global maximum
   - Solution: Random restart, backtracking

2. PLATEAU:
   - Flat area with same value neighbors
   - Solution: Big jumps, random moves

3. RIDGE:
   - Steep area but narrow passage
   - Solution: Try multiple directions
```

Q4 (a) Explain state space search with example. (5 marks)

Answer:

```
State Space Search:

Definition: Represent problem as graph of states with operators

Components:
1. Initial State: Where problem starts
2. Goal State: Desired end state
3. Operators: Actions that change state
4. State Space: All reachable states

Example: 8-Puzzle

Initial:       Goal:
1 2 3          1 2 3
4 5 6    →     4 5 6
7 0 8            7 8

State: [1,2,3,4,5,6,7,0,8]

Operators:
- Slide blank up, down, left, right
- Each move creates new state
- Search for path from initial to goal
```

Q4 (b) Differentiate between BFS and DFS. (5 marks)

Aspect	BFS	DFS
Data Structure	Queue	Stack
Strategy	Level by level	Deep first
Optimal	Yes	No
Complete	Yes	No (can loop)
Time	O(b^d)	O(b^m)
Space	O(b^d)	O(bm)
Path	Shortest	First found

Question Paper - 2

Q1 (a) What is knowledge-based agent? Explain with diagram. (3 marks)

```
Knowledge-Based Agent:

An agent that uses knowledge representation and reasoning.

┌──────────────────────────────────────┐
│         Knowledge-Based              │
│              Agent                   │
├──────────────────────────────────────┤
│  ┌──────────────┐  ┌───────────────┐ │
│  │  Knowledge   │  │   Inference   │ │
│  │    Base      │  │    Engine     │ │
│  └──────────────┘  └───────────────┘ │
└──────────────────────────────────────┘

Components:
- KB: Stores facts and rules
- Inference Engine: Derives new conclusions
- TELL: Adds knowledge
- ASK: Queries knowledge
```

Q1 (b) What is Wumpus World? Explain its characteristics. (3 marks)

```
Wumpus World:

A grid-based problem demonstrating knowledge-based agents

Characteristics:
- Partially Observable (only local perception)
- Deterministic (exact outcomes)
- Not episodic (sequential)
- Static (wumpus doesn't move)
- Discrete states
- Single agent

Sensors: Stench, Breeze, Glitter, Bump, Scream
Actuators: Forward, Turn Left, Turn Right, Grab, Release, Shoot
```

Q1 (c) Explain forward and backward chaining with example. (4 marks)

```
Forward Chaining (Data-Driven):
- Start with facts
- Apply rules to derive conclusions
- Continue until goal reached

Example: KB = {King(John), Greedy(John), King(x)∧Greedy(x)→Evil(x)}
Step 1: Facts: {King(John), Greedy(John)}
Step 2: Apply rule → Infer Evil(John)
Step 3: Goal Evil(John) reached ✓

Backward Chaining (Goal-Driven):
- Start with goal
- Work backwards to find facts
- More focused

Example: Goal = Evil(John)
Step 1: Find rule with Evil(x)
Step 2: Need King(John) ∧ Greedy(John)
Step 3: Both are facts ✓
```

Q2 (a) Explain resolution with steps. (5 marks)

```
Resolution - Theorem Proving:

Steps:
1. Convert all sentences to CNF (Conjunctive Normal Form)
2. Negate the conclusion
3. Add to knowledge base
4. Resolve pairs of clauses
5. If empty clause (⊥) derived → Contradiction → PROVED!

Resolution Rule:
(L₁ ∨ L₂ ∨ ... ∨ Lₙ) and (¬L₁ ∨ M)
──────────────────────────────────────
L₂ ∨ ... ∨ Lₙ ∨ M

Example:
KB: ¬P ∨ Q, P
Goal: Q

1. Add ¬Q to KB
2. ¬P ∨ Q + P → Q
3. Q + ¬Q → ⊥ (EMPTY CLAUSE)

Proved! ✓
```

Q2 (b) Explain A* algorithm with properties. (5 marks)

```
A* Search Algorithm:

Formula: f(n) = g(n) + h(n)

Properties:

1. ADMISSIBLE:
   h(n) never overestimates true cost
   h(n) ≤ h*(n)

2. OPTIMAL:
   Finds minimum cost path with admissible h

3. COMPLETE:
   Will find solution if exists

4. CONSISTENT (for graph search):
   h(n) ≤ cost(n,n') + h(n')

Time Complexity: O(b^d)
Space Complexity: O(b^d)

Example: Shown in Q2(a) above
```

Additional Previous Year Questions & Answers

Unit 1: Additional Questions

Q1 (i) Describe the difference between Weak AI and Strong AI. (2 marks)

Answer:

Weak AI (Narrow AI)	Strong AI (Artificial General Intelligence)
Designed for specific tasks	Can perform any intellectual task
Limited to programmed functions	Can understand, learn, adapt
No consciousness or understanding	Possesses genuine understanding
Examples: Siri, Alexa, Chess programs	Theoretical concept
Cannot generalize beyond domain	Can transfer learning to new domains

Key Differences:

```
Weak AI:
- Focuses on specific, narrow applications
- Works within predetermined boundaries
- Simulates intelligence but doesn't "think"
- Examples: Image recognition, language translation

Strong AI:
- Possesses human-level intelligence
- Can reason, learn, understand context
- Has general cognitive abilities
- Currently theoretical (not achieved)
```

Q1 (ii) Explain the rule-based approach in production systems. (3 marks)

Answer:

Rule-Based Approach in Production Systems:

A production system uses **IF-THEN rules** (productions) to represent knowledge and drive problem-solving.

```
Production Rule Format:
IF <condition> THEN <action>

Example:
IF (car won't start AND battery is dead) THEN (replace battery)
```

Components of Rule-Based Production System:

Component	Description
Production Rules	Set of IF-THEN rules representing knowledge
Working Memory	Contains current facts about the problem
Inference Engine	Applies rules to working memory
Conflict Resolution	Decides which rule to apply when multiple match

How It Works:

```
1. Match: Compare conditions in rules with facts in working memory
2. Select: Choose best rule (conflict resolution)
3. Execute: Perform action specified by rule
4. Update: Modify working memory with new facts
5. Repeat until goal reached or no rules apply
```

Example: Medical Diagnosis

```
Rule 1: IF fever AND cough THEN cold
Rule 2: IF fever AND rash THEN measles
Rule 3: IF cold AND duration > 10 THEN pneumonia

Working Memory: {fever, cough, duration = 5}
→ Rule 1 matches → Add cold to working memory
→ Check if goal reached
```

Q1 (iii) Difference between declarative intelligence and procedural intelligence (2 marks)

Answer:

Aspect	Declarative Intelligence	Procedural Intelligence
Knowledge Form	Facts and statements	Skills and procedures
Representation	"Knows that"	"Knows how to"
Storage	Explicit facts in memory	Implicit sequences of actions
Acquisition	Told or read	Practice and experience
Example	"Paris is the capital of France"	How to ride a bicycle
Processing	Retrieval	Execution

Detailed Explanation:

Declarative Intelligence:

- Knowledge about facts and relationships
- Can be stated explicitly
- Easy to communicate and share
- Stored as propositions

- Examples:
  - "Water freezes at 0°C"
  - "The Earth revolves around the Sun"
  - "All mammals are warm-blooded"

Procedural Intelligence:

- Knowledge about how to do things
- Skills and expertise
- Difficult to articulate
- Stored as action sequences/behaviors
- Examples:
  - Riding a bicycle
  - Playing piano
  - Driving a car
  - Solving equations

Q1 (iv) What are the model-based intelligent agents? Explain with architectural diagram. (3 marks)

Answer:

Model-Based Intelligent Agents:

A model-based agent maintains an **internal state** that represents the world, allowing it to handle partially observable environments.

```
┌──────────────────────────────────────────────────────────────┐
│                    MODEL-BASED REFLEX AGENT                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌──────────────┐     ┌──────────────┐     ┌──────────────┐  │
│    │   PERCEPT    │────►│   AGENT      │────►│   ACTION     │  │
│    │   SENSOR     │     │   PROGRAM    │     │  ACTUATOR    │  │
│    └──────────────┘     └──────────────┘     └──────────────┘  │
│                                  ↓                           │
│                          ┌──────────────┐                    │
│                          │    STATE     │                    │
│                          │  (Internal   │                    │
│                          │   Model)     │                    │
│                          └──────────────┘                    │
│                                  ↓                           │
│                          ┌──────────────┐                    │
│                          │    WORLD     │                    │
│                          │   MODEL      │                    │
│                          │  (How world  │                    │
│                          │   works)     │                    │
│                          └──────────────┘                    │
└──────────────────────────────────────────────────────────────┘
```

Components:

Component	Function
Percept	Input from sensors
Internal State	Current representation of world
World Model	Knowledge about how actions affect world
Transition Model	Predicts next state given current state and action
Sensor Model	Predicts what sensors would perceive

How It Works:

```
1. Percept received from environment
2. Update internal state using:
   - Current percept
   - Previous state
   - World model (how actions affect state)
3. Consult world model to choose best action
4. Execute action via actuators
5. Repeat

Example: Self-driving car
- Sensors: Cameras, LIDAR, GPS
- Internal State: Position, velocity, nearby objects
- World Model: Physics of driving, traffic rules
- Actions: Accelerate, brake, steer
```

Advantages:

- Handles partially observable environments
- Can reason about unobserved aspects
- More intelligent than simple reflex agents

Unit 2: Additional Questions

Q2 (a) Explain the following terms with example: Supervised Learning, Unsupervised Learning, Reinforcement Learning (5 marks)

Answer:

1. Supervised Learning

```
Definition: Learning from labeled data (input-output pairs)
Goal: Learn a mapping from inputs to outputs

Types:
├── Classification: Discrete output (cat/dog)
└── Regression: Continuous output (price prediction)

Example - Email Spam Detection:
┌──────────────────────────────────────────┐
│  TRAINING DATA                           │
│  Email 1: "Free money!" → SPAM ✓         │
│  Email 2: "Meeting at 3pm" → NOT SPAM ✓    │
│  Email 3: "Click here to win" → SPAM ✓    │
│                                          │
│  LEARNED MODEL:                          │
│  IF contains "free" AND "money" → SPAM      │
│  IF contains "meeting" → NOT SPAM           │
└──────────────────────────────────────────┘
```

Algorithm Examples:

- Decision Trees
- Neural Networks
- Support Vector Machines
- Linear Regression
- Naive Bayes

---

2. Unsupervised Learning

```
Definition: Learning from unlabeled data
Goal: Find patterns, structure, or relationships

Types:
├── Clustering: Group similar items (K-means)
├── Dimensionality Reduction: Reduce features (PCA)
└── Association: Find rules (Apriori)

Example - Customer Segmentation:
┌──────────────────────────────────────────┐
│  INPUT: Customer purchase history         │
│  Customer A: [milk, bread, eggs]          │
│  Customer B: [diapers, baby food]         │
│  Customer C: [milk, bread, butter]         │
│                                            │
│  DISCOVERED PATTERNS:                     │
│  Cluster 1 (Family): diapers, baby food   │
│  Cluster 2 (Daily): milk, bread, eggs     │
│  Cluster 3 (Daily Plus): milk, bread...   │
└──────────────────────────────────────────┘
```

Algorithm Examples:

- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- Autoencoders

---

3. Reinforcement Learning

```
Definition: Learning through interaction with environment
Goal: Maximize reward through trial and error

Key Components:
├── Agent: Learner/decision maker
├── Environment: What agent interacts with
├── Action: What agent can do
├── Reward: Feedback from environment
└── Policy: Strategy for choosing actions

Example - Learning to Play Chess:

Episode 1:
  Agent plays random moves
  Loses game → Reward: -1
  Learns: Don't make that move again

Episode 100:
  Agent makes better moves
  Wins game → Reward: +1
  Learns: This move sequence is good

Episode 1000:
  Agent plays like grandmaster!
  Consistent wins → Optimal policy learned

┌──────────────────────────────────────────┐
│  ENVIRONMENT                              │
│  ┌─────────┐                             │
│  │  Chess  │ ← Agent takes actions       │
│  │  Board  │ → Receives reward/penalty   │
│  └────┬────┘                             │
│       ↓                                   │
│  Reward: +1 (win), -1 (loss), 0 (draw)   │
└──────────────────────────────────────────┘
```

Algorithm Examples:

- Q-Learning
- Deep Q-Network (DQN)
- Policy Gradient Methods
- Actor-Critic Methods

Q2 (b) What do you mean by Hill Climbing? Explain the limitations associated with this algorithm. (5 marks)

Answer:

Hill Climbing Algorithm:

```
Hill Climbing is a local search algorithm that:
- Starts from an initial state
- Iteratively moves to a better neighbor state
- Uses gradient ascent (for maximization) or descent (for minimization)
- Continues until no better neighbor is found (local maximum/minimum)

Basic Algorithm:
1. Start with initial state
2. Evaluate current state
3. Generate all neighbor states
4. If any neighbor is better, move to best neighbor
5. Else, stop (local optimum reached)
```

```python
"""
Hill Climbing Algorithm - Detailed Explanation

WHAT IT DOES:
- Local search optimization
- Greedy approach - always moves to better state
- No backtracking - commits to each move
- Used for optimization problems
"""

def hill_climbing(problem):
    """
    Hill Climbing Algorithm

    Args:
        problem: Object with:
                 - initial_state()
                 - get_successors()
                 - value(state) / heuristic(state)

    Returns:
        Best state found (local optimum)
    """
    # Step 1: Start with initial state
    current = problem.initial_state()

    while True:
        # Step 2: Get all neighbor states
        neighbors = current.get_successors()

        # Step 3: Find best neighbor
        best_neighbor = None
        best_value = current.value()

        # Step 4: Evaluate all neighbors
        for neighbor in neighbors:
            if neighbor.value() > best_value:
                best_neighbor = neighbor
                best_value = neighbor.value()

        # Step 5: If no better neighbor, we're done
        # Current state is local maximum
        if best_value <= current.value():
            return current

        # Step 6: Move to better neighbor
        current = best_neighbor
```

LIMITATIONS OF HILL CLIMBING:

Limitation	Description	Example
Local Maximum	Stuck at peak that's not the global peak	Finding average solution instead of best
Plateau	Flat area where all neighbors have same value	No improvement possible
Ridge	Steep path to peak but narrow	Cannot reach in single move
Slow Convergence	Takes many iterations to improve	Large search space
No Guarantee	May miss better solutions	Not complete

---

1. LOCAL MAXIMUM:

```
Problem: Stuck at local peak instead of global peak

Solution Graph:
                    Global Maximum (Best!)
                         ★
                        / \
                       /   \
                      /     \
            Local    /       \
            Max ★    \       /
                      \     /
                       \   /
                        \ /

When climbing: From left, we reach local max and STOP
              But global max is on the right!
```

Solutions:

- Random restart: Start from multiple random points
- Simulated annealing: Occasionally accept worse moves
- Local beam search: Track multiple states

---

2. PLATEAU:

```
Problem: Flat area where all states have same value

         ┌──────────────────────────────┐
         │         PLATEAU              │
         │   ★ ★ ★ ★ ★ ★ ★ ★ ★ ★     │
         │   (all same value)           │
         └──────────────────────────────┘
              ↓
          Cannot improve!
```

Solutions:

- Big jump: Make large random move
- Sideways move: Allow moving to same-value states

---

3. RIDGE:

```
Problem: Diagonal ridge - steep but narrow

          ★ (Goal)
         /|
        / |
       /  |
      /   | ← Cannot reach in single direction change
     /    |
    /     |
   ● (Current Position)

Single moves can't reach peak directly
```

Solutions:

- Try multiple directions
- Bidirectional search

Resolution Proof Questions

Q3 (a) From the given set of facts prove using Resolution: (5 marks)

Problem:

```
Anyone passing his history exams and winning the lottery is happy.
But anyone who studies or is lucky can pass all his exams.
Shree did not study but he is lucky.
Anyone who is lucky wins the lottery.
Is Shree happy?
```

Answer:

Step 1: Convert to First-Order Logic (FOL)

Let:

- P(x): x passes history exams
- W(x): x wins lottery
- H(x): x is happy
- S(x): x studies
- L(x): x is lucky

Knowledge Base:

1. ∀x ((P(x) ∧ W(x)) → H(x)) [Pass & Win → Happy]
2. ∀x ((S(x) ∨ L(x)) → P(x)) [Study OR Lucky → Pass]
3. ¬S(Shree) ∧ L(Shree) [Shree didn't study, Shree is lucky]
4. ∀x (L(x) → W(x)) [Lucky → Win]

Goal: H(Shree) [Is Shree happy?]

Step 2: Convert to CNF (Conjunctive Normal Form)

```
1. ¬P(x) ∨ ¬W(x) ∨ H(x)
2. ¬S(x) ∨ P(x)        [from S(x) ∨ L(x) → P(x)]
3. ¬L(x) ∨ P(x)
4. ¬S(Shree)           [from fact 3]
5. L(Shree)            [from fact 3]
6. ¬L(x) ∨ W(x)
```

Step 3: Negate the Goal

```
¬H(Shree)   [We want to prove Shree is happy]
```

Step 4: Resolution Proof

```
Clause 1: ¬P(x) ∨ ¬W(x) ∨ H(x)
Clause 2: ¬H(Shree)               [Negated goal]
──────────────────────────────────────────────
Resolve H(x) with ¬H(Shree):
Unifier: {x/Shree}
Result: ¬P(Shree) ∨ ¬W(Shree)    ...(A)

Clause 3: ¬S(x) ∨ P(x)
Clause 4: ¬S(Shree)
──────────────────────────────────────────────
These don't resolve directly. Let's use different approach.

Let's resolve (A) with other clauses:

(A): ¬P(Shree) ∨ ¬W(Shree)

Clause 6: ¬L(x) ∨ W(x)
With (A) by substituting x with Shree:
¬P(Shree) ∨ ¬L(Shree)          ...(B)

Clause 5: L(Shree)
Clause (B): ¬P(Shree) ∨ ¬L(Shree)
──────────────────────────────────────────────
Resolve L(Shree):
Result: ¬P(Shree)                ...(C)

Clause 2: ¬S(Shree) ∨ P(Shree)
Clause 4: ¬S(Shree)
──────────────────────────────────────────────
We have: ¬S(Shree) and ¬S(Shree) ∨ P(Shree)
This doesn't help directly.

Let's use: ¬S(x) ∨ P(x) with fact ¬S(Shree):
Since ¬S(Shree) is true, we can infer P(Shree) is true? No.

Actually: From ¬S(x) ∨ P(x) and ¬S(Shree), we CANNOT infer P(Shree)!
(Since ¬S(x) ∨ P(x) means: if NOT S(x) then P(x) is not guaranteed)

Let's use Clause 3: ¬L(x) ∨ P(x)
With L(Shree):
Clause 3: ¬L(x) ∨ P(x)
Clause 5: L(Shree)
──────────────────────────────────────────────
Resolve: {x/Shree}
Result: P(Shree)                  ...(D)

Now we have: P(Shree) from (D)

Clause (C): ¬P(Shree)
Clause (D): P(Shree)
──────────────────────────────────────────────
Resolve: EMPTY CLAUSE ⊥

CONTRADICTION!
Therefore, our assumption ¬H(Shree) is FALSE
∴ H(Shree) is TRUE!

SHREE IS HAPPY! ✓
```

Q3 (b) Prove using Propositional Logic: (5 marks)

Problem:

```
P ∧ Q = P ∨ Q
P → (Q → P)
¬(P → Q)
P → (Q ∨ R) and (P → Q) ∨ (P → R) are logically equivalent
```

Answer:

Part 1: P ∧ Q ≡ P ∨ Q ← Wait, this is NOT equivalent!

Let me solve correctly:

(i) P ∧ Q ≡ P ∨ Q

```
TRUTH TABLE VERIFICATION:

P | Q | P∧Q | P∨Q
--+---+-----+----
T | T |  T  |  T   ✓
T | F |  F  |  T   ✗
F | T |  F  |  T   ✗
F | F |  F  |  F   ✓

NOT EQUIVALENT! P∧Q ≠ P∨Q

Correct equivalence: P ∧ Q ≡ ¬(¬P ∨ ¬Q)  [De Morgan's]
```

(ii) Prove: P → (Q → P)

```
Using Implication: P → Q ≡ ¬P ∨ Q

P → (Q → P)
= P → (¬Q ∨ P)
= ¬P ∨ (¬Q ∨ P)
= ¬P ∨ P ∨ Q          [Associativity]
= TRUE ∨ Q             [¬P ∨ P = TRUE]
= TRUE

∴ P → (Q → P) is a TAUTOLOGY (always true)
```

Truth Table Verification:

```
P | Q | Q→P | P→(Q→P)
--+---+-----+----------
T | T |  T  |    T
T | F |  T  |    T
F | T |  F  |    T
F | F |  T  |    T

All TRUE → TAUTOLOGY ✓
```

(iii) Prove: ¬(P → Q) ≡ P ∧ ¬Q

```
Using Implication: P → Q ≡ ¬P ∨ Q

¬(P → Q)
= ¬(¬P ∨ Q)                  [Implication]
= ¬¬P ∧ ¬Q                   [De Morgan's]
= P ∧ ¬Q                     [Double Negation]

∴ EQUIVALENT ✓
```

(iv) Prove: P → (Q ∨ R) ≡ (P → Q) ∨ (P → R)

```
Left Side: P → (Q ∨ R)
= ¬P ∨ (Q ∨ R)                       [Implication]
= (¬P ∨ Q) ∨ R                       [Associativity]

Right Side: (P → Q) ∨ (P → R)
= (¬P ∨ Q) ∨ (¬P ∨ R)               [Implication]
= ¬P ∨ Q ∨ ¬P ∨ R                   [Associativity]
= ¬P ∨ Q ∨ R

Both sides equal: ¬P ∨ Q ∨ R

∴ EQUIVALENT ✓
```

Additional Long Answer Questions

Q4 (a) Compare forward chaining and backward chaining with an example

Answer:

Aspect	Forward Chaining	Backward Chaining
Direction	Facts → Goal	Goal → Facts
Approach	Bottom-up	Top-down
Control	Data-driven	Goal-driven
Starting Point	Known facts	Goal/conclusion
Inference	Apply rules forward	Work backwards
Derives	All possible conclusions	Only relevant facts
Efficiency	May derive irrelevant facts	More focused
Used In	Expert systems, CLIPS	Prolog, logic programming

Example:

Knowledge Base:

```
Facts:
- King(John)
- Greedy(John)

Rule:
- King(x) ∧ Greedy(x) → Evil(x)
```

Forward Chaining:

```
Step 1: Start with facts: {King(John), Greedy(John)}
Step 2: Check rule: King(x) ∧ Greedy(x) → Evil(x)
        - Can we apply? YES! Both premises satisfied.
Step 3: Derive conclusion: Evil(John)
Step 4: Check goal: Is Evil(John)? YES! ✓
```

Backward Chaining:

```
Goal: Prove Evil(John)

Step 1: Find rule with Evil(x) in conclusion
        Found: King(x) ∧ Greedy(x) → Evil(x)

Step 2: Need to prove: King(John) ∧ Greedy(John)
        - Is King(John) a fact? YES! ✓
        - Is Greedy(John) a fact? YES! ✓

Step 3: Goal PROVED! ✓
```

Q4 (b) Implement Best First Search and find best solution with cost. J is goal state.

Answer:

```
Graph: (J is goal)

        S
       /|\
      / | \
     A  B  C
    /|  |  |\
   D E  F  G H
   |  |  |  |
   I  J  J  J

Costs:
S→A=3, S→B=2, S→C=4
A→D=4, A→E=2
B→F=1, B→I=6
C→G=3, C→H=5
D→I=3
E→J=5
F→J=2
G→J=4
H→J=6

Heuristic h(n): (estimated distance to J)
h(S)=6, h(A)=5, h(B)=4, h(C)=3
h(D)=4, h(E)=5, h(F)=2, h(G)=4, h(H)=6, h(J)=0
```

```python
"""
Best First Search Algorithm - Implementation

KEY CONCEPT:
- Expands node with LOWEST heuristic value h(n)
- Uses priority queue ordered by h(n)
- NOT guaranteed to find optimal path
"""

import heapq

def best_first_search(graph, start, goal, heuristic):
    """
    Best First Search Algorithm

    Args:
        graph: Dictionary with adjacency and costs
               Format: {'A': [('B', 5), ('C', 2)]}
        start: Starting node
        goal: Goal node
        heuristic: Dictionary of h(n) values

    Returns:
        Path from start to goal
    """
    # Step 1: Initialize priority queue with start node
    # Priority = heuristic value (lower = better)
    priority_queue = [(heuristic[start], start, [start])]
    visited = set()

    # Step 2: Main loop
    while priority_queue:
        # Step 3: Get node with LOWEST h(n) value
        h_value, node, path = heapq.heappop(priority_queue)

        # Step 4: Skip if visited
        if node in visited:
            continue

        # Step 5: Mark as visited
        visited.add(node)

        # Step 6: Check if goal
        if node == goal:
            return path

        # Step 7: Explore neighbors
        for neighbor, cost in graph.get(node, []):
            if neighbor not in visited:
                # Add based on heuristic value
                heapq.heappush(priority_queue,
                              (heuristic[neighbor], neighbor,
                               path + [neighbor]))

    return None


# Graph definition
graph = {
    'S': [('A', 3), ('B', 2), ('C', 4)],
```

```
    'A': [('D', 4), ('E', 2)],
    'B': [('F', 1), ('I', 6)],
    'C': [('G', 3), ('H', 5)],
    'D': [('I', 3)],
    'E': [('J', 5)],
    'F': [('J', 2)],
    'G': [('J', 4)],
    'H': [('J', 6)],
    'I': [],
    'J': []
}

heuristic = {
    'S': 6, 'A': 5, 'B': 4, 'C': 3,
    'D': 4, 'E': 5, 'F': 2, 'G': 4, 'H': 6, 'J': 0
}

# Execute
result = best_first_search(graph, 'S', 'J', heuristic)
print(f"Best First Search Path: {result}")

# Output shows path based on heuristic, NOT necessarily optimal cost!
```

Q5 (a) What is heuristic function? Explain its components.

Answer:

Heuristic Function h(n):

A heuristic is a **rule of thumb** or **informed guess** that helps guide search toward the goal. It's a function that estimates the cost from any node n to the goal.

```
h(n) = Estimated cost from node n to goal
```

Components of Heuristic:

Component	Description
Domain Knowledge	Information specific to the problem
Estimate	Approximate remaining cost
Admissible	Never overestimates true cost
Consistent	For graph search: h(n) ≤ cost(n,n') + h(n')

How Heuristic Works:

```
In A* Search: f(n) = g(n) + h(n)

┌──────────────────────────────────────────────┐
│           f(n) = g(n) + h(n)              │
│                                              │
│  g(n): Actual cost from START to n         │
│         (known, computed)                    │
│                                              │
│  h(n): Estimated cost from n to GOAL       │
│         (heuristic, guessed)                 │
│                                              │
│  f(n): Total estimated cost                  │
│         (combined estimate)                  │
└──────────────────────────────────────────────┘
```

Example - 8 Puzzle:

```
Heuristic: Number of misplaced tiles

Current State:     Goal State:
1 2 3              1 2 3
4 5 6     →        4 5 6
7 0 8              0 7 8

Misplaced tiles: 2 (7 and 8 are wrong)
h(n) = 2

This heuristic is ADMISSIBLE because:
- Each misplaced tile needs at least 1 move
- Never overestimates actual moves needed
```

Q5 (b) How production system useful for solving AI problems?

Answer:

Production Systems for AI Problem Solving:

A production system provides a **framework for representing and applying knowledge** to solve problems.

Why Production Systems are Useful:

Benefit	Explanation
Modularity	Rules are independent, easy to add/remove
Modifiability	Change rules without changing program
Knowledge Separation	Domain knowledge separate from control
Incremental	Add knowledge gradually
Natural	IF-THEN format mirrors human reasoning
Traceable	Can explain reasoning steps

Example Applications:

```
1. EXPERT SYSTEMS:
   Rule: IF temperature > 100 AND duration > 10min
         THEN diagnose("Fever")

2. DIAGNOSTIC SYSTEMS:
   Rule: IF engine_won't_start AND battery_dead
         THEN action("replace_battery")

3. CONFIGURATION SYSTEMS:
   Rule: IF computer_type = "gaming" AND budget > 1000
         THEN recommend("RTX GPU")
```

Components Working Together:

```
┌──────────────────────────────────────────────┐
│         PRODUCTION SYSTEM                    │
├──────────────────────────────────────────────┤
│                                              │
│  ┌──────────────┐                            │
│  │   RULES      │  IF-THEN knowledge         │
│  │   (KB)       │  "Production memory"       │
│  └──────────────┘                            │
│         │                                    │
│         ↓                                    │
│  ┌──────────────┐                            │
│  │   MATCHER    │  Find applicable rules     │
│  │  (Inference) │  Pattern matching          │
│  └──────────────┘                            │
│         │                                    │
│         ↓                                    │
│  ┌──────────────┐                            │
│  │  CONFLICT    │  When multiple rules       │
│  │ RESOLUTION   │  apply, choose one         │
│  └──────────────┘                            │
│         │                                    │
│         ↓                                    │
│  ┌──────────────┐                            │
│  │  EXECUTOR    │  Apply selected rule       │
│  │   (ACT)      │  Update working memory     │
│  └──────────────┘                            │
│         ↓                                    │
│  ┌──────────────┐                            │
│  │    WORKING   │  Current facts/state       │
│  │    MEMORY    │  "Database"                │
│  └──────────────┘                            │
└──────────────────────────────────────────────┘
```

Q5 (c) What is AI? How do you compare human intelligence with artificial intelligence?

Answer:

What is Artificial Intelligence?

```
AI is the study of making machines capable of performing
tasks that would require intelligence if done by humans.

Categories:
1. Systems that think like humans (Cognitive approach)
2. Systems that act like humans (Turing Test)
3. Systems that think rationally (Logic approach)
4. Systems that act rationally (Rational Agent approach)
```

Comparison: Human Intelligence vs Artificial Intelligence

Aspect	Human Intelligence	Artificial Intelligence
Basis	Biological (brain)	Electronic (computer)
Learning	Few examples, self-learning	Needs many examples
Processing	Parallel (many things at once)	Sequential (mostly)
Storage	Associative, fuzzy	Address-based, exact
Reasoning	Intuitive, creative	Algorithmic, logical
Consciousness	Self-aware	No awareness
Adaptability	Highly adaptive	Limited to training
Errors	Fatigue, emotional	Hardware, software bugs
Speed	Slower for calculations	Faster for computations
Accuracy	Prone to mistakes	Highly accurate

Key Differences:

```
Human Intelligence:
- Can understand context and nuance
- Makes judgment calls based on experience
- Learns from single example
- Creative and innovative
- Has emotions and consciousness

Artificial Intelligence:
- Follows programmed rules
- Needs explicit data for patterns
- Requires massive training data
- Optimizes within constraints
- No emotions or consciousness
```

Q5 (d) How is unification used in resolution? Explain with an example.

Answer:

Unification in Resolution:

Unification is the process of **finding substitutions** that make two logical expressions identical. In resolution, it's used to **match complementary literals** in clauses.

Unification Algorithm:

```python
"""
Unification - Finding substitutions to make expressions match

KEY CONCEPTS:
1. SUBSTITUTION: {variable/term}
2. UNIFIER: Makes two expressions identical
3. MGU: Most General Unifier (simplest substitution)
4. OCCURS CHECK: Prevents infinite terms
"""

def unify(x, y, theta=None):
    """
    Unification Algorithm

    Returns substitution making x and y identical
    """
    if theta is None:
        theta = {}

    # 1. If identical, return substitution
    if x == y:
        return theta

    # 2. If x is variable, bind to y
    if is_variable(x):
        return unify_var(x, y, theta)

    # 3. If y is variable, bind to x
    if is_variable(y):
        return unify_var(y, x, theta)

    # 4. If both compound, check predicate
    if is_compound(x) and is_compound(y):
        if get_predicate(x) != get_predicate(y):
            return None  # Different predicates!
        return unify(get_args(x), get_args(y), theta)

    return None
```

Example in Resolution:

```
Problem: Prove King(John) ∧ Greedy(John) → Evil(John)

Given:
1. ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
2. King(John)
3. Greedy(John)

Step 1: Negate conclusion: ¬Evil(John)

Step 2: Resolution with Clause 1 and negated goal

Clause 1: ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
Clause 2: ¬Evil(John)

Resolution:
- Complementary literals: Evil(x) and ¬Evil(John)
- Unify with θ = {x/John}
- Result: ¬King(John) ∨ ¬Greedy(John)

Step 3: Resolution with result and facts

Clause 3: King(John)
Clause 4: ¬King(John) ∨ ¬Greedy(John)

Resolution:
- Complementary literals: King(John) and ¬King(x)
- Unify with θ = {x/John}
- Result: ¬Greedy(John)

Step 4: Resolution with fact

Clause 5: Greedy(John)
Clause 6: ¬Greedy(John)

Resolution:
- Complementary literals: Greedy(John) and ¬Greedy(John)
- Unify with θ = {}
- Result: EMPTY CLAUSE ⊥

CONTRADICTION → PROOF COMPLETE!
```

Q5 (e) Why is learning considered an important component of intelligence? Give example.

Answer:

Learning as Important Component:

Learning is **fundamental to intelligence** because:

Reason	Explanation
Adaptation	Agents must adapt to changing environments
Autonomy	Reduces reliance on pre-programmed knowledge
Improvement	Performance improves with experience
Generalization	Apply knowledge to new situations
Discovery	Find patterns humans might miss

Without Learning:

- Fixed behavior, cannot improve
- Cannot handle novel situations
- Requires explicit programming for everything

With Learning:

- Improves over time
- Adapts to user preferences
- Discovers new patterns
- Becomes more efficient

Example: Learning in AI

```
1. SUPERVISED LEARNING - Learning from examples

   Input: Images labeled as "cat" or "dog"
   Process: Algorithm learns features
   Output: Can classify new images

2. REINFORCEMENT LEARNING - Learning from rewards

   Agent plays chess:
   - Win → +1 reward
   - Lose → -1 reward
   - Over time, learns optimal strategy

3. UNSUPERVISED LEARNING - Learning patterns

   Input: Customer purchase histories
   Process: Find natural groupings
   Output: Customer segments for marketing
```

More Short Answer Questions

Q6 (a) What is an intelligent agent? Explain its different types.

Answer:

Intelligent Agent:

An intelligent agent is an entity that **perceives its environment through sensors** and **acts upon it through actuators** to achieve goals.

Types of Agents:

Type	Description	Example
Simple Reflex	Responds to current percept	Thermostat
Model-Based	Maintains internal state	Self-driving car
Goal-Based	Has goals to achieve	Chess computer
Utility-Based	Maximizes utility	Robot planner
Learning Agent	Improves from experience	AlphaGo

Q6 (b) Various problems encountered in Hill Climbing.

Answer:

Problem	Description	Solution
Local Maximum	Peak lower than global	Random restart
Plateau	Flat region	Random jump
Ridge	Steep but narrow	Multiple directions
Slow Progress	Many iterations	Better heuristic

Q7 (a) Represent facts as predicates (Marcus Story)

Answer:

Facts as Predicates:

```
1. Marcus was a man.
   man(Marcus)

2. Marcus was a Pompeian.
   pompeian(Marcus)

3. All Pompeians were Romans.
   ∀x (pompeian(x) → roman(x))

4. Caesar was a ruler.
   ruler(caesar)

5. All Romans were either loyal to Caesar or hated him.
   ∀x (roman(x) → loyal(caesar, x) ∨ hates(x, caesar))

6. Everyone is loyal to someone.
   ∀x ∃y loyal(y, x)

7. People only try to assassinate rulers they are not loyal to.
   ∀x ∀y (ruler(y) ∧ ¬loyal(y, x) → tries(x, y, assassinate))

8. Marcus tried to assassinate Caesar.
   tries(marcus, caesar, assassinate)
```

Q7 (b) Differentiate between declarative and procedural knowledge

Answer:

Aspect	Declarative	Procedural
Form	Facts, statements	Actions, procedures
"Knows that"	✓	
"Knows how"		✓
Storage	Explicit	Implicit
Examples	Rules, facts	Skills, habits

Additional Syllabus Topics (Previously Missing)

Unit 2: Additional Knowledge Representation Topics

1. Knowledge Representation Issues

What is Knowledge Representation?

Knowledge Representation (KR) is the study of how knowledge can be represented and used by AI systems.

Key Issues in Knowledge Representation:

Issue	Description	Challenge
Representation Accuracy	How to accurately represent real-world knowledge	Must handle ambiguity, uncertainty
Inferential Adequacy	Ability to derive new knowledge	Must support reasoning
Inferential Efficiency	Speed of reasoning	Must be computationally feasible
Expressiveness	What can be expressed	Trade-off with complexity
Acquisition	How to get knowledge into system	Manual vs automatic
Maintenance	Updating knowledge	Handling contradictions

---

2. Mapping in Knowledge Representation

What is Mapping?

Mapping in KR refers to the relationship between:

- Real-world concepts and symbolic representations
- Objects in the world and symbols in the system

Types of Mapping:

Type	Description	Example
Symbolic Mapping	Direct correspondence	"Cat" → cat()
Structural Mapping	Based on relationships	isa(cat, animal)
Hierarchical Mapping	Taxonomic relationships	taxonomy tree
Associative Mapping	Related concepts	associations

Example:

```
Real World:          Knowledge Base:
┌──────────────┐        ┌──────────────────┐
│   Object   │──────→│   Predicate/Atom  │
│  (Cat)     │       │   isa(cat, animal)│
└──────────────┘        └──────────────────┘
```

---

3. Frame Problem

What is the Frame Problem?

The Frame Problem is one of the fundamental problems in AI knowledge representation. It refers to the difficulty of representing what **doesn't change** in a dynamic world when actions are performed.

Why is it a Problem?

In the real world, when we take an action, MOST things stay the same (the "frame"). But traditional logic makes us explicitly state ALL the things that don't change, which is computationally expensive.

```python
"""
THE FRAME PROBLEM - Explained with Code Example

When an action is performed, we need to represent:
1. What CHANGES (effects)
2. What STAYS THE SAME (frame axioms)

Without frame problem:
- We must explicitly list every unchanged property
- "After robot moves, robot's color is still red"
- "After robot moves, robot's battery level is still 80%"
- ... millions of such statements!

With frame problem solution:
- Only list what CHANGES
- Assume everything else stays the same
"""

# Example: Robot World State
class WorldState:
    """
    Represents the state of the world
    Each attribute must be tracked explicitly without frame problem solution
    """
    def __init__(self):
        # These ALL need to be tracked!
        self.robot_location = "room_A"
        self.robot_color = "red"
        self.robot_battery = 80
        self.door_status = "closed"
        self.light_status = "on"
        # ... imagine hundreds of such attributes!

    def move_robot(self, from_room, to_room):
        """
        Move robot - must update EVERYTHING that changes
        """
        self.robot_location = to_room  # This changes

        # PROBLEM: Must also remember everything that DOESN'T change!
        # Without frame reasoning, we must explicitly state:
        self.robot_color = self.robot_color   # Stays same
        self.robot_battery = self.robot_battery - 5  # Changes
        # ... imagine 1000 more lines!


# SOLUTION: Frame Axioms
"""
Frame Axiom approach:
Instead of listing what stays the same, we use rules like:
"IF robot moves from A to B THEN robot_location = B"
"IF robot moves from A to B THEN robot_color stays same"
"IF robot moves from A to B THEN robot_battery decreases by 5"

This is still inefficient!
"""

# Better Solution: Event Calculus
class EventCalculus:
    """
```

```python
    Event Calculus - A solution to the frame problem

    Key concepts:
    - Fluents: Properties that can change over time
    - Events: Actions that can occur
    - HoldsAt(fluent, time): Is a property true at a time?
    - Initiates(event, fluent, time): Event starts a property
    - Terminates(event, fluent, time): Event ends a property
    """

    def __init__(self):
        # Store fluents as (property, value) at time
        self.fluents = {}  # {(fluent_name, time): value}

    def holds_at(self, fluent, time):
        """Check if fluent is true at given time"""
        return self.fluents.get((fluent, time), False)

    def initiates(self, event, fluent, time, value):
        """When event occurs, fluent becomes true"""
        self.fluents[(fluent, time)] = value

    def terminates(self, event, fluent, time):
        """When event occurs, fluent becomes false"""
        self.fluents[(fluent, time)] = False


# Example usage
ec = EventCalculus()

# Initial state (time 0)
ec.initiates(None, "robot_at", 0, "room_A")
ec.initiates(None, "robot_color", 0, "red")
ec.initiates(None, "robot_battery", 0, 80)

# Robot moves at time 1
# Only specify what CHANGES:
ec.terminates("move_robot", "robot_at", 1)
ec.initiates("move_robot", "robot_at", 1, "room_B")
ec.initiates("move_robot", "robot_battery", 1, 75)

# We DON'T NEED to specify:
# - robot_color stays red (ASSUMED!)
# - door stays closed (ASSUMED!)
# etc.

# Query: What is robot color at time 1?
print(f"Robot color at time 1: {ec.holds_at('robot_color', 1)}")  # "red" - assumed!
```

Solutions:

Solution	Description	Example
Frame Axioms	Explicitly state what doesn't change	"After move, color stays same"
Event Calculus	Use fluents and time points	HoldsAt(property, time)
Situation Calculus	Represent states as situations	Result(action, state)
Default Reasoning	Assume nothing changes unless stated	Closed world assumption
Frame Axioms	Explicitly state what doesn't change	
Event Calculus	Use fluents and time points	
Situation Calculus	Represent states as situations	
Default Reasoning	Assume nothing changes unless stated	

---

4. Predicate Logic (Extended)

Predicate Logic (First-Order Logic) extends propositional logic with:

Element	Example	Description
Predicates	likes(x, y)	Relations between objects
Variables	x, y, z	Represent objects
Quantifiers	∀, ∨	"For all", "There exists"
Functions	father(x)	Map objects to objects

Syntax:

```
Term → constant | variable | function(term, ...)
Atomic → predicate(term, ...)
Sentence → Atomic | ¬Sentence | Sentence ∧ Sentence | ∀x Sentence | ∃x Sentence
```

---

5. Facts in Logic

Facts are basic assertions in logic that are assumed to be true.

Types of Facts:

Type	Example	Representation
Atomic Facts	"Socrates is a man"	man(socrates)
Simple Facts	"All cats are mammals"	∀x (cat(x) → mammal(x))
Complex Facts	"John likes Mary and Mary likes John"	likes(john, mary) ∧ likes(mary, john)

Facts vs Rules:

```
Facts:     Assertions that are TRUE
           Example: man(socrates), mortal(socrates)

Rules:     IF-THEN relationships
           Example: ∀x (man(x) → mortal(x))
```

---

6. Instance and Isa Relationship

Instance Relationship (isa):

The "instance" or "isa" relationship represents that an object is a member of a category.

```
Syntax: instance(object, category)
        or: isa(object, category)

Examples:
- isa(socrates, person)
- isa(cat, animal)
- isa(chair, furniture)

Reading: "socrates is a person"
         "cat is an animal"
```

Isa Hierarchy (Taxonomy):

```
              Thing
               |
          ┌────┴────┐
        living   non-living
          |
      ┌───┴───┐
    animal  plant
      |
   ┌──┴──┐
  cat   dog

Represented as:
isa(cat, animal)
isa(animal, living)
isa(living, thing)
```

Properties of Isa:

- Transitive: If isa(A, B) and isa(B, C), then isa(A, C)
- Not symmetric: isa(A, B) ≠ isa(B, A) usually

---

7. Matching in Production Systems

What is Matching?

Matching is the process of checking if the condition (LHS) of a production rule matches facts in working memory.

```python
"""
Matching in Production Systems

KEY CONCEPT:
- Pattern matching between rule conditions and working memory
- Finds which rules are applicable
- Can be expensive (NP-complete in worst case)
"""

def match(condition, working_memory):
    """
    Match rule condition against working memory

    Args:
        condition: Pattern to match (can contain variables)
        working_memory: Set of known facts

    Returns:
        Substitution if match succeeds, None otherwise
    """
    # Simple matching algorithm

    # 1. If condition is a simple fact
    if is_atom(condition):
        return {} if condition in working_memory else None

    # 2. If condition has variables
    if has_variables(condition):
        return unify(condition, working_memory)

    return None


# Example
working_memory = {
    'man(socrates)',
    'mortal(socrates)',
    'likes(john, mary)'
}

# Match rule: ?x is man → ?x is mortal
# Condition: man(?x)
# Match: man(socrates) with ?x = socrates
# Result: {?x/socrates}
```

Types of Matching:

Type	Description
Exact Match	Condition matches exactly
Partial Match	Some variables bound
Pattern Match	Contains wildcards/variables
Rete Match	Efficient algorithm for many rules

---

8. Control Knowledge

What is Control Knowledge?

Control knowledge is knowledge about **how to use** the knowledge base - the "meta-knowledge" about reasoning.

```
Knowledge Base:
- Facts: What is true
- Rules: Relationships between facts

Control Knowledge:
- Which rule to apply first?
- What order to explore?
- When to stop?
- How to handle conflicts?
```

Examples:

Control Strategy	Description
Rule Ordering	Apply rules in specific order
Conflict Resolution	When multiple rules apply, choose which
Search Control	Which path to explore first
Meta-rules	Rules about rules

Control Knowledge in Production Systems:

```
Example Conflict Resolution Strategies:
1. Specificity: Apply most specific rule first
2. Recency: Apply rule using most recent facts
3. Priority: User-defined priorities
4. Random: Random selection
```

---

9. Symbolic Reasoning Under Uncertainty

What is Symbolic Reasoning Under Uncertainty?

Handling reasoning when information is incomplete, unreliable, or probabilistic.

Sources of Uncertainty:

Type	Example
Incomplete Information	Not all facts known
Noisy Data	Sensor errors
Ambiguous Language	"He" could refer to multiple people
Inconsistent Knowledge	Conflicting rules

Approaches:

Approach	Description	Example
Probability Theory	Bayesian networks, probabilistic logic	P(A|B) = 0.8
Fuzzy Logic	Degrees of truth (0-1)	temperature is "warm" = 0.7
Dempster-Shafer	Belief functions	Bel(A) + Uncert(A) = 1
Default Reasoning	Assume typical case	"Birds fly" unless proven otherwise

---

10. Non-Monotonic Reasoning

What is Non-Monotonic Reasoning?

Non-Monotonic Reasoning is reasoning where conclusions can be **retracted** when new information is added. This is crucial in everyday human reasoning where we make assumptions that can be overturned.

```python
"""
NON-MONOTONIC REASONING - Complete Explanation

KEY CONCEPT:
- In monotonic logic: Adding knowledge NEVER removes conclusions
- In non-monotonic logic: Adding knowledge CAN remove conclusions

Real-world example:
- "Birds fly" - We assume true initially
- "Penguin is a bird" - We assume penguins can't fly
- "Tweety is a penguin" - We RETRACT our earlier conclusion!

This is how humans naturally reason!
"""

# ==================== IMPLEMENTATION ====================

class NonMonotonicReasoner:
    """
    A simple non-monotonic reasoner using defaults and exceptions
    """

    def __init__(self):
        # Facts: things we know are definitely true
        self.facts = set()

        # Default rules: "Normally X" unless exception
        # Format: (category, property, default_value, exceptions)
        self.defaults = {}

        # Exception rules
        self.exceptions = {}

    def add_fact(self, fact):
        """Add a definite fact to knowledge base"""
        self.facts.add(fact)

    def add_default(self, category, property_name, default_value):
        """
        Add a default rule

        Example: add_default('bird', 'flies', True)
        Means: Normally birds fly
        """
        self.defaults[category] = {
            'property': property_name,
            'default': default_value,
            'exceptions': []
        }

    def add_exception(self, category, exception_category, property_name, value):
        """
        Add an exception to a default

        Example: add_exception('bird', 'penguin', 'flies', False)
        Means: Penguins (exception to birds) don't fly
        """
        if category in self.defaults:
            self.defaults[category]['exceptions'].append({
                'exception_type': exception_category,
```

```python
                'property': property_name,
                'value': value
            })

    def query(self, entity, property_name):
        """
        Query if entity has property - uses non-monotonic reasoning
        """
        # Step 1: Check if we have direct fact
        fact = f"{property_name}({entity})"
        if fact in self.facts:
            return True, "Direct fact"

        # Step 2: Check exceptions (specific first!)
        for cat, default_info in self.defaults.items():
            if default_info['property'] == property_name:
                # Check if entity is an exception
                for exc in default_info['exceptions']:
                    # Check if entity belongs to exception category
                    if self._check_category(entity, exc['exception_type']):
                        return exc['value'], f"Exception: {exc['exception_type']}"

        # Step 3: Apply default (if no exception applies)
        for cat, default_info in self.defaults.items():
            if default_info['property'] == property_name:
                # Check if entity belongs to this category
                if self._check_category(entity, cat):
                    return default_info['default'], f"Default: {cat}"

        return None, "Unknown"

    def _check_category(self, entity, category):
        """Check if entity is in category"""
        # Check direct fact
        if f"isa({entity}, {category})" in self.facts:
            return True
        # Check parent categories (inheritance)
        return False


# ==================== EXAMPLE USAGE ====================

# Create reasoner
reasoner = NonMonotonicReasoner()

# Step 1: Add default facts
reasoner.add_fact("isa(tweety, bird)")
reasoner.add_fact("isa(penguin, bird)")
reasoner.add_fact("isa(polly, bird)")
reasoner.add_fact("isa(tweety, penguin)")

# Step 2: Add default rules
reasoner.add_default('bird', 'flies', True)
reasoner.add_default('bird', 'has_feathers', True)

# Step 3: Add exceptions
reasoner.add_exception('bird', 'penguin', 'flies', False)

# Step 4: Query!
print("=== Non-Monotonic Reasoning Queries ===")
```

```python
# Query 1: Does Tweety fly?
# Tweety is a penguin, penguins don't fly (exception!)
result, reason = reasoner.query('tweety', 'flies')
print(f"Tweety flies: {result} (Reason: {reason})")

# Query 2: Does Polly fly?
# Polly is a bird, no exception → default applies
result, reason = reasoner.query('polly', 'flies')
print(f"Polly flies: {result} (Reason: {reason})")

# Query 3: Does Tweety have feathers?
result, reason = reasoner.query('tweety', 'has_feathers')
print(f"Tweety has feathers: {result} (Reason: {reason})")


"""
KEY INSIGHT:
- When we add "isa(tweety, penguin)", the conclusion about Tweety FLIES changes!
- This is NON-MONOTONIC: Adding information REMOVED a conclusion

Compare with MONOTONIC:
- Once we know Tweety flies, adding more info can NEVER make us conclude Tweety doesn't fly
"""


# ==================== TRUTH MAINTENANCE SYSTEM ====================

class TruthMaintenanceSystem:
    """
    TMS tracks which conclusions depend on which assumptions
    Can retract conclusions when assumptions change
    """

    def __init__(self):
        # Node storage: {node_id: {'belief': bool, 'justification': [...]}}
        self.nodes = {}
        self.rules = []  # (antecedent, consequent)

    def add_belief(self, node_id, belief, justification=None):
        """
        Add a belief with justification
        """
        self.nodes[node_id] = {
            'belief': belief,
            'justification': justification or [],
            'supporters': []
        }

    def add_rule(self, antecedent_nodes, consequent_node):
        """
        Add inference rule: IF antecedent THEN consequent
        """
        self.rules.append((set(antecedent_nodes), consequent_node))

        # Add consequent with this rule as justification
        if consequent_node not in self.nodes:
            self.nodes[consequent_node] = {
                'belief': False,
                'justification': [],
                'supporters': []
            }
```

```python
        # Add supporters
        for ant in antecedent_nodes:
            if ant in self.nodes:
                self.nodes[consequent_node]['supporters'].append(ant)

    def retract(self, node_id):
        """
        Retract a belief - must retract all beliefs that depend on it!
        This is the core of non-monotonic reasoning
        """
        if node_id not in self.nodes:
            return

        print(f"Retracting: {node_id}")
        self.nodes[node_id]['belief'] = False

        # Find all beliefs that depended on this
        to_retract = self._find_dependents(node_id)

        for node in to_retract:
            print(f"  Also retracting: {node}")
            self.nodes[node]['belief'] = False

    def _find_dependents(self, node_id):
        """Find all nodes that depend on this node"""
        dependents = []

        for consequent in self.nodes:
            if node_id in self.nodes[consequent].get('supporters', []):
                dependents.append(consequent)
                dependents.extend(self._find_dependents(consequent))

        return dependents


# Example
tms = TruthMaintenanceSystem()

# Add facts
tms.add_belief('bird(tweety)', True)
tms.add_belief('flies(tweety)', True, justification=['default_bird_fly'])

# Add rule
tms.add_rule({'bird(tweety)', 'penguin(tweety)'}, 'not_flies(tweety)')

print("Before retraction:")
print(f"  Tweety flies: {tms.nodes['flies(tweety)']['belief']}")

# New information - Tweety is a penguin!
tms.add_belief('penguin(tweety)', True)

# Retract flies(tweety) because penguin assumption is now true
tms.retract('flies(tweety)')

print("\nAfter learning Tweety is penguin:")
print(f"  Tweety flies: {tms.nodes['flies(tweety)']['belief']}")
```

Why is Non-Monotonic Reasoning Important?

Reason	Example
Real-world uncertainty	We make reasonable assumptions
Incomplete information	We don't know all facts
Open-world assumption	Absence of evidence ≠ evidence of absence
Default reasoning	"Birds fly" except penguins, ostriches...

---

11. Statistical Reasoning

What is Statistical Reasoning?

Statistical Reasoning uses probability and statistics to make inferences from data. Unlike logical certainty, it deals with degrees of belief and uncertainty.

```python
"""
STATISTICAL REASONING - Complete Implementation

Statistical reasoning allows AI to make predictions
and decisions under uncertainty using probability theory.
"""

import math
from collections import defaultdict


# ==================== BAYESIAN REASONING ====================

class BayesianReasoner:
    """
    Implements Bayesian inference for probabilistic reasoning
    Uses Bayes' Theorem: P(H|E) = P(E|H) × P(H) / P(E)
    """

    def __init__(self):
        # Prior probabilities: P(Hypothesis)
        self.priors = {}

        # Likelihoods: P(Evidence | Hypothesis)
        self.likelihoods = {}

    def set_prior(self, hypothesis, probability):
        """Set prior probability P(H)"""
        self.priors[hypothesis] = probability

    def set_likelihood(self, hypothesis, evidence, probability):
        """Set likelihood P(Evidence | Hypothesis)"""
        if hypothesis not in self.likelihoods:
            self.likelihoods[hypothesis] = {}
        self.likelihoods[hypothesis][evidence] = probability

    def posterior(self, hypothesis, evidence):
        """
        Calculate posterior probability P(H | E)
        Using Bayes' Theorem
        """
        # P(H | E) = P(E | H) × P(H) / P(E)

        # Get likelihood
        likelihood = self.likelihoods.get(hypothesis, {}).get(evidence, 0)

        # Get prior
        prior = self.priors.get(hypothesis, 0)

        # Calculate P(E) - probability of evidence
        p_evidence = 0
        for h, p_h in self.priors.items():
            p_evidence += self.likelihoods.get(h, {}).get(evidence, 0) * p_h

        # Bayes' theorem
        if p_evidence == 0:
            return 0

        posterior = (likelihood * prior) / p_evidence
        return posterior
```

```python
# ==================== EXAMPLE: MEDICAL DIAGNOSIS ====================

def medical_diagnosis_example():
    """
    Real-world example: Cancer diagnosis

    Given:
    - P(Cancer) = 0.01 (1% of population has cancer)
    - P(Positive | Cancer) = 0.99 (99% accurate test for cancer patients)
    - P(Positive | No Cancer) = 0.05 (5% false positive rate)

    Question: If you test positive, what is the actual probability you have cancer?
    """

    print("=" * 60)
    print("MEDICAL DIAGNOSIS EXAMPLE")
    print("=" * 60)

    # Create Bayesian reasoner
    bayes = BayesianReasoner()

    # Set priors
    bayes.set_prior('Cancer', 0.01)      # P(Cancer) = 1%
    bayes.set_prior('NoCancer', 0.99)     # P(No Cancer) = 99%

    # Set likelihoods
    bayes.set_likelihood('Cancer', 'Positive', 0.99)      # P(+ | Cancer)
    bayes.set_likelihood('NoCancer', 'Positive', 0.05)   # P(+ | No Cancer)

    # Calculate posterior
    prob_cancer_given_positive = bayes.posterior('Cancer', 'Positive')

    print(f"\nGiven:")
    print(f"  - P(Cancer) = 0.01 (1%)")
    print(f"  - P(+ | Cancer) = 0.99 (99% sensitive)")
    print(f"  - P(+ | No Cancer) = 0.05 (5% false positive)")
    print(f"\nIf test is POSITIVE:")
    print(f"  P(Cancer | +) = {prob_cancer_given_positive:.4f}")
    print(f"  = {prob_cancer_given_positive * 100:.2f}%")

    print(f"\n*** SURPRISING RESULT! ***")
    print(f"Even with 99% accurate test, only ~16.7% chance you actually have cancer!")
    print(f"This is because cancer is rare (1%), so most positives are false positives!")


# ==================== NAIVE BAYES CLASSIFIER ====================

class NaiveBayesClassifier:
    """
    Naive Bayes Classifier - Simple but powerful probabilistic classifier
    Assumes features are conditionally independent (naive assumption)
    """

    def __init__(self):
        self.class_priors = {}      # P(Class)
        self.feature_probs = {}      # P(Feature|Class)
        self.classes = set()

    def fit(self, X, y):
```

```
        """
        Train the classifier
        X: list of feature dictionaries
        y: list of class labels
        """
        n = len(y)

        # Calculate class priors
        for c in set(y):
            self.classes.add(c)
            self.class_priors[c] = y.count(c) / n

        # Calculate conditional probabilities
        for c in self.classes:
            self.feature_probs[c] = {}

            # Count occurrences
            feature_counts = defaultdict(lambda: defaultdict(int))
            total = 0

            for xi, yi in zip(X, y):
                if yi == c:
                    total += 1
                    for feature, value in xi.items():
                        feature_counts[feature][value] += 1

            # Calculate probabilities with Laplace smoothing
            for feature, value_counts in feature_counts.items():
                self.feature_probs[c][feature] = {}
                for value, count in value_counts.items():
                    # Laplace smoothing: add 1 to avoid zero probabilities
                    self.feature_probs[c][feature][value] = (count + 1) / (total +
len(value_counts))

    def predict(self, x):
        """
        Predict class for a single sample
        Returns: most likely class
        """
        predictions = {}

        for c in self.classes:
            # Start with log of prior
            log_prob = math.log(self.class_priors[c])

            # Add log of conditional probabilities
            for feature, value in x.items():
                if feature in self.feature_probs[c]:
                    if value in self.feature_probs[c][feature]:
                        log_prob += math.log(self.feature_probs[c][feature][value])
                    else:
                        log_prob += math.log(1 / len(self.classes))  # Unknown value

            predictions[c] = log_prob

        # Return class with highest probability
        return max(predictions, key=predictions.get)


# ==================== EXAMPLE: SPAM DETECTION ====================
```

```python
def spam_detection_example():
    """
    Example: Naive Bayes for spam detection
    """

    print("\n" + "=" * 60)
    print("SPAM DETECTION EXAMPLE")
    print("=" * 60)

    # Training data
    # Features: words in email
    X_train = [
        {'free': 1, 'win': 1, 'meeting': 0, 'project': 0},
        {'free': 1, 'win': 0, 'meeting': 1, 'project': 1},
        {'free': 0, 'win': 0, 'meeting': 1, 'project': 1},
        {'free': 1, 'win': 1, 'meeting': 0, 'project': 0},
        {'free': 0, 'win': 0, 'meeting': 1, 'project': 0},
    ]

    y_train = ['spam', 'ham', 'ham', 'spam', 'ham']

    # Train classifier
    clf = NaiveBayesClassifier()
    clf.fit(X_train, y_train)

    # Test emails
    test_emails = [
        {'free': 1, 'win': 1, 'meeting': 0, 'project': 0},  # Looks like spam
        {'free': 0, 'win': 0, 'meeting': 1, 'project': 1},  # Looks like ham
    ]

    print("\nTest Results:")
    for i, email in enumerate(test_emails):
        prediction = clf.predict(email)
        print(f"  Email {i+1}: {prediction}")


# ==================== BAYESIAN NETWORK ====================

class SimpleBayesianNetwork:
    """
    Simple Bayesian Network for probabilistic reasoning
    Represents conditional dependencies between variables
    """

    def __init__(self):
        # Nodes and their parents: {node: [parents]}
        self.parents = {}

        # CPTs (Conditional Probability Tables): {node: {parent_values: probability}}
        self.cpts = {}

    def add_node(self, node, parents=None):
        """Add a node with its parents"""
        self.parents[node] = parents or []

        # Initialize CPT
        self.cpts[node] = {}

        # Calculate number of possible parent combinations
        num_combinations = 1
```

```python
        for p in self.parents[node]:
            num_combinations *= 2  # Assuming binary values

    def set_probability(self, node, parent_values, prob_true):
        """
        Set P(node=true | parent_values)
        parent_values: dict of {parent: value}
        """
        key = tuple(sorted(parent_values.items()))
        self.cpts[node][key] = prob_true

    def query(self, node, evidence=None):
        """
        Query probability of node given evidence
        Uses variable elimination (simplified)
        """
        evidence = evidence or {}

        # Sum over all possible values
        prob_true = 0
        prob_false = 0

        # Get all parent combinations
        parents = self.parents[node]

        # For each parent combination
        if not parents:
            # No parents - just return CPT
            return self.cpts[node].get((), 0.5)

        # Simplified: assume binary with no evidence
        return self.cpts[node].get((), 0.5)


# Run examples
if __name__ == "__main__":
    medical_diagnosis_example()
    spam_detection_example()
```

Statistical Reasoning Methods:

Method	Description	Use Case
Bayesian Networks	Graph of probabilistic dependencies	Medical diagnosis, fault detection
Naive Bayes	Simple classifier with independence assumption	Spam filtering, text classification
Markov Models	Sequential probability models	Speech recognition, time series
Hidden Markov Models	Unobserved state sequences	NLP, gesture recognition
Monte Carlo	Sampling-based inference	Complex probability calculations
MONOTONIC: Adding premises never removes conclusions		

```
If KB ⊨ A, then KB ∪ {B} ⊨ A
```

NON-MONOTONIC: Adding premises CAN remove conclusions New information may invalidate old conclusions

```
**Example:**
```

Initial Knowledge:

1. Bird(Tweety)
2. normally_birds_fly(Tweety) Conclusion: Flies(Tweety) [DEFAULT - assumed true]

New Information Added: 3. Penguin(Tweety) 4. penguins_dont_fly(Tweety)

Conclusion RETRACTED: Flies(Tweety) is NO LONGER true!

Non-Monotonic Reasoning Systems:

System	Description
Default Logic	Use defaults unless contradicted
Circumscription	Assume minimal models
Autoepistemic Logic	Beliefs about beliefs
Frame-Based	Exceptions to inheritance

---

11. Statistical Reasoning

What is Statistical Reasoning?

Using probability and statistics to make inferences from data.

Key Concepts:

Concept	Formula	Description
Bayes' Theorem	P(A|B) = P(B|A) × P(A) / P(B)	Conditional probability
Prior Probability	P(A)	Belief before evidence
Posterior Probability	P(A|B)	Belief after evidence
Likelihood	P(B|A)	Probability of evidence given hypothesis

Statistical Reasoning Methods:

Method	Description
Bayesian Networks	Directed acyclic graphs of probabilistic dependencies
Naive Bayes	Simple probabilistic classifier
Markov Models	Sequential probability models
Hidden Markov Models	Unobserved state models

```python
"""
Example: Bayesian Reasoning

P(Disease|Symptoms) = P(Symptoms|Disease) × P(Disease) / P(Symptoms)

# Given:
P(Disease) = 0.01          # 1% of population has disease
P(Positive|Disease) = 0.99 # Test 99% accurate for diseased
P(Positive|No Disease) = 0.05 # 5% false positive

# Calculate P(Disease|Positive):
P(Positive) = 0.99×0.01 + 0.05×0.99 = 0.0594

P(Disease|Positive) = (0.99 × 0.01) / 0.0594 = 0.167

# Only 16.7% chance actually has disease!
```

Unit 1: Additional Topics

1.1 AI Foundations

The Foundations of Artificial Intelligence:

Field	Contribution to AI
Philosophy	Logic, mind, consciousness
Mathematics	Logic, algorithms, computability
Psychology	Cognitive processes, behaviorism
Linguistics	Language structure, NLP
Computer Science	Hardware, algorithms
Neuroscience	Neural networks, brain models
Economics	Game theory, decision theory
Biology	Evolutionary algorithms, genetic algorithms

1.2 AI Techniques

Technique	Description
Search	Blind and heuristic search
Knowledge Representation	Facts, rules, frames
Reasoning	Logical inference
Learning	From data/experience
Planning	Sequence of actions
Perception	Vision, speech recognition

1.3 AI Models

Model	Description
Symbolic AI	Rule-based, logic systems
Connectionist AI	Neural networks
Evolutionary AI	Genetic algorithms
Bayesian AI	Probabilistic models
Hybrid AI	Combination of above

1.4 Search Methods and Issues

Search Method Issues:

Issue	Description	Solution
Combinatorial Explosion	Too many states	Heuristics, pruning
Local Minima	Stuck in suboptimal solution	Random restart, annealing
Cycle Detection	Repeating states	Visited list
Memory	Too much storage	Iterative deepening
Optimality	Finding best solution	Admissible heuristics
Completeness	Finding any solution	Complete algorithms

End of Document

Notes prepared according to AI 302T Syllabus Including: Complete Theory, Algorithms with Code Comments, Examples, All Previous Year Questions with Detailed Solutions